Journal of Network and Computer Applications 249 (2026) 104455

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Research paper

Alert prediction in computer networks using transformer-based temporal graph neural networks: Identifying the next victim

Zahra Makki Nayeri, Mohsen Rezvani *

Faculty of Computer Engineering at Shahrood University of Technology, Shahrood, Iran

ARTICLE INFO

Dataset link: https://github.com/ZahraMakkiNayeri/TGNE-TA.git

Keywords:
Alert prediction
Deep graph learning
Temporal graph neural networks
Dynamic deep neural networks
Transformer-based encoder
Link prediction
Temporal link prediction
Multi-class link prediction

ABSTRACT

Despite significant advancements in intrusion detection systems, cyber-attacks continue to increase in severity and complexity, making proportionate reactive responses insufficient to mitigate their impact. While prior studies have focused on estimating attack likelihood or predicting individual steps of multi-stage attacks, accurately predicting attacker–victim interactions and the corresponding attack type within specified timeframes remains a challenging and underexplored problem. In this work, we propose a dynamic graph-based deep learning framework for alert prediction. We first employ a Temporal Graph Network (TGN) to model evolving attacker–victim interactions and perform binary link prediction. However, standard TGN architectures rely on simple message aggregation mechanisms and are inherently limited to binary outcomes. To address this limitation, we extend TGN to TGNE, an edge-classification model that enables multiclass prediction of attack types. To further enhance temporal reasoning, we introduce TGNE-TA, which replaces the conventional aggregation scheme with a Transformer-based message aggregator. Although Transformer models employ positional encoding, this mechanism represents only discrete order information and does not capture real temporal intervals or irregular time gaps. Therefore, we incorporate a Dual-Level Temporal Encoding mechanism to enable the model to capture both global and local temporal dependencies more effectively. The proposed framework constructs attack graphs from alert streams using sliding time windows and learns temporal patterns through deep neural networks. Experiments on real-world datasets demonstrate that TGNE-TA achieves over 90% accuracy in multiclass prediction and consistently outperforms eight state-of-the-art deep learning baselines. These results highlight the effectiveness of the proposed approach in forecasting future attacks, identifying attacker–victim interactions, and predicting associated attack categories.

1. Introduction

The detection and prevention of malicious activities in computer networks remain critical challenges in the field of cybersecurity. Traditional intrusion detection systems (IDS) typically rely on predefined attack signatures or observable anomalies to identify potential threats. However, these reactive approaches often fall short when it comes to preventing sophisticated, evolving cyber-attacks. Recent research has highlighted the need for proactive, predictive strategies that can anticipate malicious activities before they escalate into disruptive events (Bartos et al., 2019). While the field of cyber-attack detection has seen significant progress, the development of predictive models for forecasting future attacks has lagged, though it has garnered increasing attention in recent years (Bartos et al., 2019).

Consider an operational scenario in which a Computer Emergency Response Team (CERT) receives an alert indicating that a specific database server is likely to be targeted by a reconnaissance scan within the next 30 min. Armed with this prediction, the team can proactively restrict access to sensitive services, closely monitor suspicious traffic patterns, or deploy targeted firewall rules before the attack materializes (Husák et al., 2018). Such early interventions can significantly reduce the attack surface, shorten response time, and prevent escalation into more severe intrusions. This example highlights the transformative potential of predictive cyber threat intelligence, shifting security operations from reactive incident response toward proactive defense.

Most existing research on alert prediction has focused on forecasting the occurrence of future attacks or predicting subsequent stages in multi-step attack sequences (Qin and Lee, 2004; Liu, 2020; Melis et al., 2019; Bartos et al., 2019; Husák and Kašpar, 2018; Husák et al., 2020). Common approaches leverage probabilistic models, data mining techniques, and basic machine learning algorithms (Husák et al., 2020). While tasks such as blacklist generation and prediction of future behavior from known malicious sources provide valuable insights, they also

* Corresponding author.
E-mail addresses: zmakki@shahroodut.ac.ir (Z. Makki Nayeri), mrezvani@shahroodut.ac.ir (M. Rezvani).

https://doi.org/10.1016/j.jnca.2026.104455
Received 21 July 2025; Received in revised form 2 February 2026; Accepted 20 February 2026
Available online 27 February 2026
1084-8045/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

exhibit notable limitations. For instance, blacklisting all IP addresses previously associated with malicious activities is often impractical due to firewall limitations and the high potential for false positives. These challenges necessitate more granular predictive methods capable of generating refined blacklists or enacting targeted defenses, such as limiting login attempts. Additionally, critical open questions remain in areas such as understanding attackers' communication strategies for coordinating attacks and predicting potential victims (Ansari et al., 2022).

Early efforts between 2005 and 2008 explored attack graphs as tools for attack planning and intrusion detection (Hughes and Sheyner, 2003). In 2013, Chang et al. introduced the NICE system, which employed attack graphs to model and mitigate attacks within virtual networks (Chung et al., 2013), while Kotenko and Chechulin developed CAMIAC to model cyber-attacks and assess their impact (Kotenko and Chechulin, 2013). Although these systems provided valuable contributions to attack planning, they did not incorporate machine learning nor did they attempt to predict attacker-victim relationships. More recently, Li et al. used system log data to detect and predict attacks through sequential log analysis and memory models, generating attack graphs but not forecasting the connections between attackers and victims (Li et al., 2023).

Despite these promising developments, there remains a significant gap in the ability to predict the attacker-victim interactions and the attack type simultaneously (Husák et al., 2020). Addressing this gap is a crucial research problem that, once solved, could significantly enhance the effectiveness of proactive cybersecurity defenses. Our approach enables not only the identification of which IP (or host) is likely to be attacked next, but also what kind of attack (e.g., reconnaissance, DDoS, anomaly) is expected. This fine-grained prediction is directly actionable for CERT and SOC teams (Husák et al., 2018).

In this paper, we propose a dynamic graph-based deep learning framework for alert prediction, built upon the Temporal Graph Network (TGN). We begin with the base TGN model to predict the likelihood of an attack's occurrence. We then extend this framework to Temporal Graph Network-Edge classification (TGNE), enabling not only alert prediction but also identification of attacker-victim interactions, prediction of future victims, and classification of attack types (e.g. Scan). Finally, we incorporate a Transformer-based aggregator (TGNE-TA) with a novel Dual-Level Temporal Encoding framework that combines local interaction time with global temporal position, enhancing temporal representation learning in TGN-based models.

Our approach leverages dynamic graph deep learning to model communication patterns between network entities, capturing temporal dependencies within the interaction sequences. Specifically, the model employs a temporal graph neural network to anticipate potential attack interactions.

Our approach is evaluated against state-of-the-art deep learning techniques reviewed in previous research (Ansari et al., 2022), which effectively model the associations between attack elements and capture long-term dependencies. Experimental results show that our temporal graph neural network model outperforms existing methods, achieving an alert prediction accuracy over 90% in real-world datasets, with lower average prediction errors.

The main contributions of this paper are as follows:

- We developed a predictive model that leverages dynamic graph representations to predict attacks based on alerts generated from known attack sources.
- We introduced a Transformer-based aggregator for TGN-based models.
- We propose a novel Dual-Level Temporal Encoding framework that integrates local (relative) and global (absolute) temporal signals, improving the temporal expressiveness of TGN-based models.
- Our approach utilizes graph-based modeling to forecast future victim IP addresses based on historical interactions between attacker and victim IPs.
- The model enables simultaneous target and source IP interaction prediction by applying a temporal graph neural network.
- Additionally, our method accurately predicts attack types based on identified malicious sources.

The remainder of this paper is organized as follows: Section 2 covers the preliminaries and threat model. Section 3 introduces the proposed model and its architecture, while Section 4 presents the experimental setup and results. Section 5 reviews the research problem alongside existing solutions. Section 6 provides a discussion, and outlines limitations and future work. Finally, Section 7 concludes the paper.

2. Preliminaries and threat model

This section outlines the key concepts and foundational information necessary for a comprehensive understanding of the remainder of the paper.

2.1. Temporal graph neural network

Dynamic graphs are typically categorized into two main types: discrete-time dynamic graphs, which consist of sequences of static graph snapshots taken at specific intervals in time, providing a step-by-step view of the network's evolution; and continuous-time dynamic graphs, which offer a more detailed and flexible representation by recording events as they occur. These events can include the addition or deletion of edges, the introduction or removal of nodes, and transformations of node or edge features, allowing for a continuous and nuanced depiction of the dynamic network (Rossi et al., 2020).

We used a temporal multigraph modeled as a sequence of time-stamped events $G = \{x(t_1), x(t_2), \ldots\}$, representing the addition or modification of nodes or interactions between node pairs at times $0 \leq t_1 \leq t_2 \leq \ldots$. An event $x(t)$ can be of two types: (1) A node-wise event, represented by $v_i(t)$, where i denotes the node index and v is the attribute vector associated with the event. If the index i is new, the event creates node i with the specified features; otherwise, it updates the existing node's features. (2) An interaction event between nodes i and j is represented by a (directed) temporal edge $e_{ij}(t)$. Since there can be multiple edges between node pairs, this is technically a multigraph. We define $V(T) = \{i : \exists v_i(t) \in G, t \in T\}$ and $E(T) = \{(i,j) : \exists e_{ij}(t) \in G, t \in T\}$ as the temporal sets of vertices and edges, respectively, and $N_i(T) = \{j : (i,j) \in E(T)\}$ as the neighborhood of node i in the time interval T. The k-hop neighborhood is denoted by $N_i^k(T)$. A snapshot of the temporal graph G at time t is the multigraph $G(t) = (V[0,t], E[0,t])$ with $n(t)$ nodes (Rossi et al., 2020).

Temporal Graph Network: A neural model for dynamic graphs functions as an encoder–decoder pair. The encoder maps the dynamic graph to node embeddings, while the decoder uses these embeddings to make task-specific predictions, such as edge prediction (Rossi et al., 2020).

Based on Rossi et al. (2020), we employ a Temporal Graph Network (TGN) encoder for a continuous-time dynamic graph, represented as a sequence of time-stamped events. As illustrated in Fig. 1, this encoder produces node embeddings Z for each time t, $Z(t) = (z_1(t), \ldots, z_{n(t)}(t))$.

2.2. Alert prediction

Definition 1 (*Alert Prediction*)**.** In cybersecurity, alert prediction is defined as the proactive identification and forecasting of potential security alerts or warnings within a network environment. These alerts indicate unusual activities, potential threats, or signs of malicious behavior that may compromise network integrity, such as unauthorized access attempts, malware infections, or data exfiltration. By leveraging

Table 1
A sample of typical Warden alerts.

Detect time	Flow count	Source IP	Target IP	Port	Protocol	Category
2019-03-11T00:05:00+02:00	17 094	185.192.59.136	142.252.135.136	22	TCP	Recon scan
2019-03-12T00:45:00+02:00	5113	78.234.46.141	142.252.32.63	443	TCP	Availability Dos
2019-03-14T00:25:00+02:00	15	185.192.59.136	142.252.32.63	81	TCP	Availability DDos
2019-03-14T00:25:00+02:00	39	78.234.46.141	142.252.32.63	22	UDP	Anomaly.Traffic

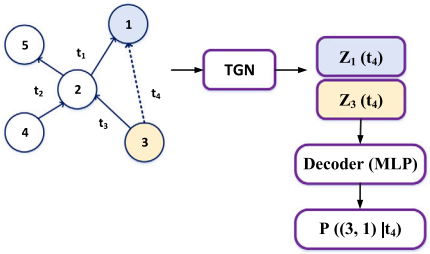


Fig. 1. Temporal graph network encoder.

historical security data and applying machine learning techniques, alert prediction aims to identify patterns that precede these warnings, allowing for timely intervention and enabling security teams to take preventative actions before an incident escalates.

2.2.1. Alert data

In this paper, we utilize the Warden alert dataset, which was continuously collected on the SABU platform over one week, from March 11 to March 17, 2019 (Husák et al., 2020). Warden, also known as the SABU platform,[1] is an alert data and threat-sharing system operated by CESNET, the Czech National Research and Education Network (Kacha et al., 2015). Table 1 presents a portion of the data frame along with its features. This study's alert features include Detect Time, Flow Count, Source IP, Target IP, Port, Protocol, and Category. Detect Time captures the moment an alert is generated. Flow Count measures attack intensity. Source IP is the attacker IP and Target IP is the victim IP. Port specifies the accessed victim's port, offering clues about exploited vulnerabilities, while Protocol highlights the communication method (e.g., TCP, UDP) used during the attack. Category represents the attack type (e.g. Reconnaissance, Scan, Attempt Exploit, etc.) and is a critical predictor for future attack patterns. We only had access to one week of network alert data. Access to longer timeframes was restricted and not publicly available. Consequently, we also utilized a second dataset, NF-UNSW-NB15-v2, to complement our analysis and examine the robustness of our model under a different data distribution.

2.3. Threat model and assumptions

To ensure tractable and focused modeling of temporal attacker-victim interactions, we adopt a bounded adversarial model tailored to the properties of the evaluated datasets. The following assumptions define the operational threat context considered in this work:

1. Limited Knowledge Adversary: The adversary does not have knowledge of the underlying prediction algorithms or learned model parameters. This ensures the integrity of learned behavior patterns and avoids adversarial manipulation during evaluation.
2. Unidirectional Interactions: Each interaction is assumed to represent a one-way attack flow from an attacker to a victim. Bidirectional or retaliatory attacks are not explicitly modeled. While this simplifies edge semantics, it aligns with the labeling convention in the datasets used.
3. Role Disjointness per Interaction: IP addresses play distinct roles per interaction — either as attacker or victim — but do not simultaneously assume both roles within the same event. This constraint is dataset-driven and avoids ambiguity in training signals.
4. Closed-world Attack Taxonomy: The set of attack types predicted is limited to those annotated in the dataset. Novel (zero-day) attack types are outside the current scope and are deferred to future work on open-set or zero-shot classification.
5. Known Entity Space: While the prediction framework is designed to operate over the set of IP addresses observed in the dataset, the model is trained and evaluated in both transductive and inductive settings. In the inductive setting, the model demonstrates the ability to generalize to unseen nodes (i.e., IP addresses not present during training). However, the prediction space remains bounded to entities within the dataset due to labeling availability and evaluation consistency. Future work may extend this to open-world prediction where entirely novel entities are detected and classified dynamically.

These assumptions are not intended to fully capture the diversity and complexity of real-world cyber threats, but rather to isolate the core challenge of learning temporal patterns in dynamic graphs for predictive purposes. Future extensions could explore open-world settings, overlapping node roles, and attacker adaptation by integrating adversarial learning or continual update mechanisms.

3. Proposed method

The main objective of this paper is to predict future attacks on computer networks by analyzing previous attacks that occurred in specific time windows. Given the complexity of computer networks, robust modeling is essential to capture the communication and interactions among the computers involved effectively. Graph-based methods are suitable for this task, as they can represent the network's structure and facilitate the analysis of connections to predict potential attacks. Since attacks can occur at different time intervals, we utilize temporal graph learning method to enhance prediction accuracy. In this paper, we propose a dynamic graph-based framework that progressively extends the Temporal Graph Network (TGN). As shown in Fig. 2, the framework evolves from TGN for binary link prediction, to TGNE for multi-class attacker–victim and attack-type prediction, then to TGNE-TA with a Transformer-based aggregator, and finally to TGNE-TA enhanced with Dual-Level Temporal Encoding to capture local and global temporal dynamics.

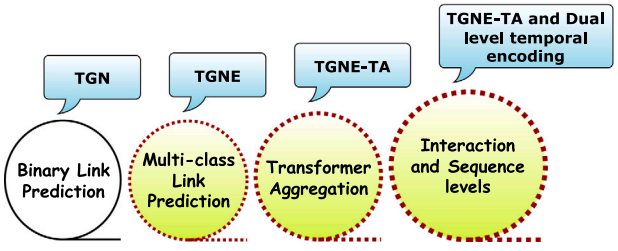


Fig. 2. Architectural evolution of the proposed framework.

[1] https://sabu.cesnet.cz

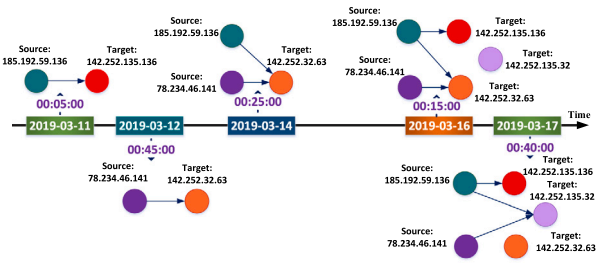


Fig. 3. Sample of a graph-based representation of alert data.

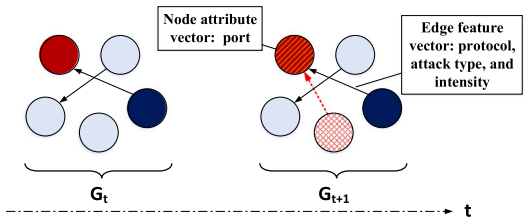


Fig. 4. The graph evolution over time (G_t to G_{t+1}), illustrating how attacker–victim interactions form temporal edges.

3.1. Graph modeling

Definition 2 (*Attack Graph*)**.** The attack graph $G = (V, E)$ is constructed from alert records. The node set V comprises attacker IPs and victim IPs, with no connections allowed within the same type (i.e., no attacker-to-attacker or victim-to-victim links). Directed edge $e(u, v)$ is created from an attacker node u to a victim node v when an attack is observed, thus capturing the asymmetry of attacks where interactions are initiated by attackers.

To enrich predictive capabilities, nodes and edges are assigned feature vectors. Node features include targeted port information, reflecting services or vulnerabilities exploited in attacks. Edge features capture the protocol, attack type, and interaction intensity between attacker and victim.

For instance, the graph-based representation of the alert records in Table 1 is illustrated in Fig. 3.

The graph presents the following properties:

1. Bipartite Graph: A graph $G = (V, E)$ is bipartite since V can be partitioned into two disjoint sets V_A (attackers) and V_V (victims), such that:

$$E \subseteq \{(u, v) \mid u \in V_A, v \in V_V\}$$

V_A contains nodes representing attackers launching alerts such as Reconscan, Anomaly Traffic, Availability DoS, and Availability DDoS. V_V contains nodes representing victims targeted by these alerts. This formulation reflects the inherently asymmetric nature of cyber-attacks, where alerts originate from attackers and target victims.

2. Multigraph: The graph $G = (V, E)$ is a multigraph since multiple edges are allowed between the same attacker $u \in V_A$ and victim $v \in V_V$. Each edge represents a unique type of alert or attack instance. Formally:

$$E = \{(u, v, i) \mid u \in V_A, v \in V_V, i \in I\}$$

where i is the identifier of the alert type (e.g., Reconscan, DoS). For example:

$$(A_1 \to V_1, Reconscan), \quad (A_1 \to V_1, DoS)$$

3. Dynamic Graph: A dynamic graph $G_t = (V_t, E_t)$ is time-variant, where the set of nodes V_t and edges E_t evolve over time based on alert timestamps. Formally:

$$E_t = \{(u, v, i, t) \mid u \in V_A, v \in V_V, i \in I, t \in T\}$$

An edge (A_1, V_1, DoS, t_1) indicates a DoS attack from A_1 to V_1 at time t_1. The graph evolves dynamically as new alerts are added with their respective timestamps.

3.2. Transform alert prediction to link prediction

As we defined, the alert prediction task can transform into a temporal link prediction problem in an attack graph. Specifically, given the state of the attack graph G_t at time t, the goal is to predict future edges in G_{t+1}, indicating potential future attacks. Fig. 4 illustrates this process, where a red edge in G_{t+1} represents a predicted attack derived from the previous structure in G_t.

3.3. General architecture

The general architecture of the proposed alert prediction methods is shown in Fig. 5. The flow begins with data loading and preprocessing, followed by constructing an attack graph to represent entities and interactions. Node and edge features are embedded, the dataset is split for evaluation, and temporal information is encoded. Finally, TGN and TGN-Edge models classify and predict attack patterns using the processed graph data.

3.3.1. Data loading and preprocessing

We prepared the dataset by loading the raw CSV data into a Pandas DataFrame, selecting key features for prediction including source IP, target IP, attack type, port, protocol, flow count, and detect time and discarding irrelevant attributes. We standardized timestamps to ISO 8601 format for time consistency, then sorted the data chronologically to capture sequential patterns essential for dynamic graph modeling and time-based predictions.

3.3.2. Attack graph construction

During the preprocessing of our bipartite graph construction stage, we convert Source and Target IPs to strings, making them hashable for downstream tasks. Next, we encode categorical features, converting Protocol, Category, and Port values into numeric indices via label encoding to make them suitable for machine learning. Each unique Source and Target IP is assigned a distinct integer ID to define unique nodes. We then extract key features like Flow Count, Port, and Protocol into feature vectors for each interaction. To maintain a bipartite structure, we offset destination node IDs by adding the maximum source node ID, ensuring separate indices for sources and destinations. Padding and zero-filled feature matrices handle any missing data, guaranteeing that each node has a complete feature vector. This graph, with rich node and edge features, is ready for modeling and analysis.

3.3.3. Node and edge feature representation

We embedded node and edge features in the bipartite graph model to capture complex interactions. Edge features like FlowCount, Protocol, and Category were embedded to provide a rich context for predicting attack patterns, while node features include Port, Protocol, and Category were similarly embedded and supplemented with edge-derived information to enhance representational depth. We select these features, since they are consistently available in real-world alert logs and capture complementary aspects of attack behavior, semantics, and intent. This comprehensive feature set for each node and edge supports more accurate predictions within the graph.

3.3.4. Data splitting

We split the data chronologically using timestamp quantiles: the 70th percentile defines $t_{\text{validation}}$ and the 85th percentile defines t_{test}. Interactions with $t \leq t_{\text{validation}}$ form the training set, those with $t_{\text{validation}} < t \leq t_{\text{test}}$ form validation, and those with $t > t_{\text{test}}$ form the test set. This preserves temporal order and prevents future information from leaking into training.

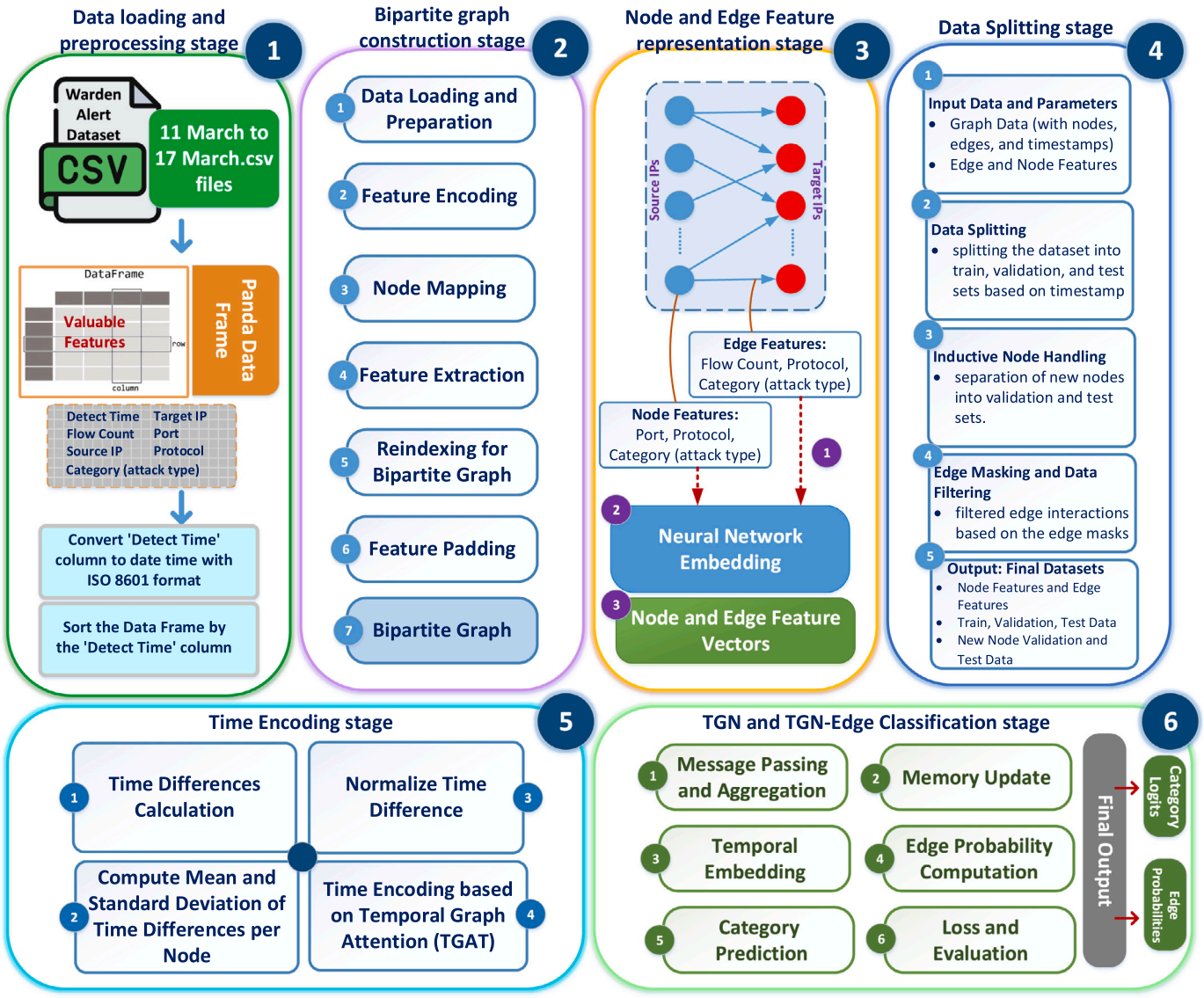


Fig. 5. An overview of TGN and TGN-Edge Classification for alert prediction flow.

For inductive evaluation, following the TGN protocol, we construct a set of new nodes. After determining $t_{\text{validation}}$, we identify nodes that first appear after this time and randomly sample 10% of all unique nodes as the new-node set. All interactions involving these nodes are removed from the training set, ensuring they are unseen during training. These nodes may appear in validation and test to assess inductive performance, while the remaining nodes form the transductive setting.

3.3.5. Time encoding

In this stage, time intervals between interactions for each entity are calculated. These time differences (intervals) are then normalized by subtracting the mean and dividing by the standard deviation. This normalization helps the model operate on a standardized temporal scale, which can prevent large time differences from distorting the temporal encoding. Then we use the time encoding presented in Time2Vec (Kazemi et al., 2019) and used in Temporal Graph Attention (Xu et al., 2020; Rossi et al., 2020).

3.3.6. TGN and TGN-edge classification with traditional aggregator

TGN is a specialized framework for learning on dynamic graphs, capturing evolving interactions between nodes over time. As illustrated in Fig. 6, the Message and Aggregation modules enable nodes to iteratively collect information from neighboring nodes, with the traditional aggregation method (last aggregator) directly impacting the precision and relevance of the resulting embeddings. The Memory Update module retains historical information for nodes. Embedding module constructs temporal embeddings by incorporating historical interactions. These embeddings are converted into edge probability scores in the Edge Probability Computation module, estimating the likelihood of connections between node pairs. Now the TGNE comes to the picture, simultaneously, the Category Prediction module applies a category predictor layer to output logits across set categories, assigning the most probable category label to each interaction. During training, the losses are combined from edge probability and category predictions to optimize the embeddings for improved classification accuracy and predictive capability.

3.3.7. TGNE computations

Fig. 6 shows computations performed by TGN and TGNE on a batch of time-stamped interactions that consists of five main modules: message, message aggregator, memory, memory update, and embedding. Each module is distinct in structure, contributing specific functionalities to the network. A key element of TGN is the Raw Message Store, which maintains the raw messages generated from past interactions. When a new batch is processed, the memory is first updated only with messages stored from previous batches. This ensures that the memory state used for prediction reflects interactions strictly earlier in time, preventing any leakage of feature information from the current batch. After updating the memory, node embeddings are computed from this updated memory state. Finally, the model stores the raw messages generated by the current batch into the Raw Message Store so that they will be used when future batches are processed (Rossi et al., 2020). In TGNE's temporal edge classification setting, all memory-related modules operate identically to the original TGN. We additionally incorporate a Combination module, which constructs edge embeddings. We applied the sum of the interaction embeddings to compute edge embeddings. Both node embeddings and combination embeddings are computed after the memory has been updated using only past information. This procedure implicitly implements a strict time-based masking: the category label and features of the current interaction are never accessible during prediction, as the raw message for the target edge is added to the store only after its prediction. The memory-related computations influence the loss of binary link prediction and therefore receive gradients. The Combination module contributes to both the binary link-prediction loss and the multi-class edge-classification loss. Importantly, binary link prediction and edge-category prediction are performed simultaneously, both conditioned on the same memory state, which contains no information from the target interaction being predicted.

3.3.8. TGN-edge classification with transformer aggregator

TGN and TGNE use simple aggregation methods (such as *last* or *mean*) to combine past messages, which limits the ability to model complex temporal dependencies.

To overcome this limitation, TGNE-TA extends TGNE by replacing the simple aggregator with a Transformer-based message aggregator that uses self-attention over historical messages to capture richer temporal patterns.

In TGNE-TA, the Memory Update module retains historical node states and is refreshed using aggregated messages obtained through the Transformer encoder. These temporal embeddings are then used for two tasks: edge probability estimation (binary link prediction) and edge category classification. The Category Prediction module applies a classifier to the edge embedding, producing logits over pre-defined edge categories. Both tasks, link prediction and edge classification, are optimized jointly during training. The losses are combined from edge probability and category predictions to optimize the embeddings for improved classification accuracy and predictive capability.

3.3.9. TGNE-TA computations

Fig. 6 illustrates the computation flow of TGNE-TA on a batch of time-stamped interactions. In the message computation step, a raw message is generated for each interaction using the features of the source and destination nodes, the edge features, and the corresponding timestamp. These messages represent latent interactions that are still context-free.

To integrate contextual and temporal dependencies among messages, we introduce a Transformer encoder as a core component of the message aggregation module. For each node, the Transformer encoder takes a sequence of its historical raw messages (typically sorted by time) as input. The encoder applies multi-head self-attention to model interactions between a node's own past messages, enabling it to selectively attend to the most relevant prior interactions. This mechanism allows the model to capture complex temporal patterns beyond simple aggregation. The learned attention weights provide qualitative insight into which past interactions contribute most to a node's current prediction.

To support temporal awareness, we inject two forms of time encoding:

- Edge-level time encoding (via a Time Encode module) to embed interaction timestamps, and
- Positional or absolute timestamp encodings within the Transformer to retain the order and spacing of messages.

The output of the Transformer encoder is an aggregated message representation for each node, summarizing its recent history in a time-aware and attention-guided manner. This aggregated message is then used to update the memory module, which stores dynamic node states. These updated memories are used to generate node embeddings. In the final stage, node embeddings are combined (by summation or concatenation) to produce an edge embedding, which is used for both link prediction (binary classification) and edge category classification (multi-class prediction). Both tasks are trained in parallel, share the same memory and embedding pipeline, but each has a separate loss function.

3.4. Time-aware encodings of TGNE-TA

To enhance the modeling of temporal dependencies and contextual information in dynamic graphs, we incorporate a Transformer-based message aggregator within the TGNE framework. Unlike traditional schemes such as mean or last aggregation, the Transformer allows the model to attend over the full sequence of past messages associated with a node.

Each message sequence is encoded using multi-head self-attention layers. To provide temporal context, we explore two types of time-aware encodings that are added to message embeddings prior to attention:

- **Temporal Encoding (Timestamp-based):** Inspired by the positional encoding in Transformers, we apply a sinusoidal encoding directly to normalized timestamps of interactions, capturing relative temporal distances between events.
- **Positional Encoding (Index-based):** The standard Transformer encoding based on message order within the sequence. This provides information about the relative position of messages, regardless of the actual time interval between them.

Empirically, we found that using standard positional encoding, which ignores actual timestamps and only captures message ordering, consistently outperformed timestamp-based temporal encoding in our experiments. This indicates that, in our settings, the relative sequence of events was more informative than the exact temporal gaps.

3.4.1. Dual-level temporal encoding

To more comprehensively model the temporal dynamics of interaction graphs, we adopt a dual-level encoding strategy, incorporating both local and global time signals:

- **Interaction-Level Encoding:** For each edge, we apply a TGAT-style learnable time encoding module to capture fine-grained temporal signals between node interactions. This encoding is concatenated with node and edge features during message computation.
- **Sequence-Level Encoding:** Within the Transformer encoder aggregator, we use positional encoding (either index-based or timestamp-based) to represent the relative order of incoming messages. This enables the Transformer to model temporal context over the message history.

This dual-level design allows the model to distinguish what happened when (edge time) and how recent or frequent events are (node history), supporting both short- and long-term temporal reasoning.

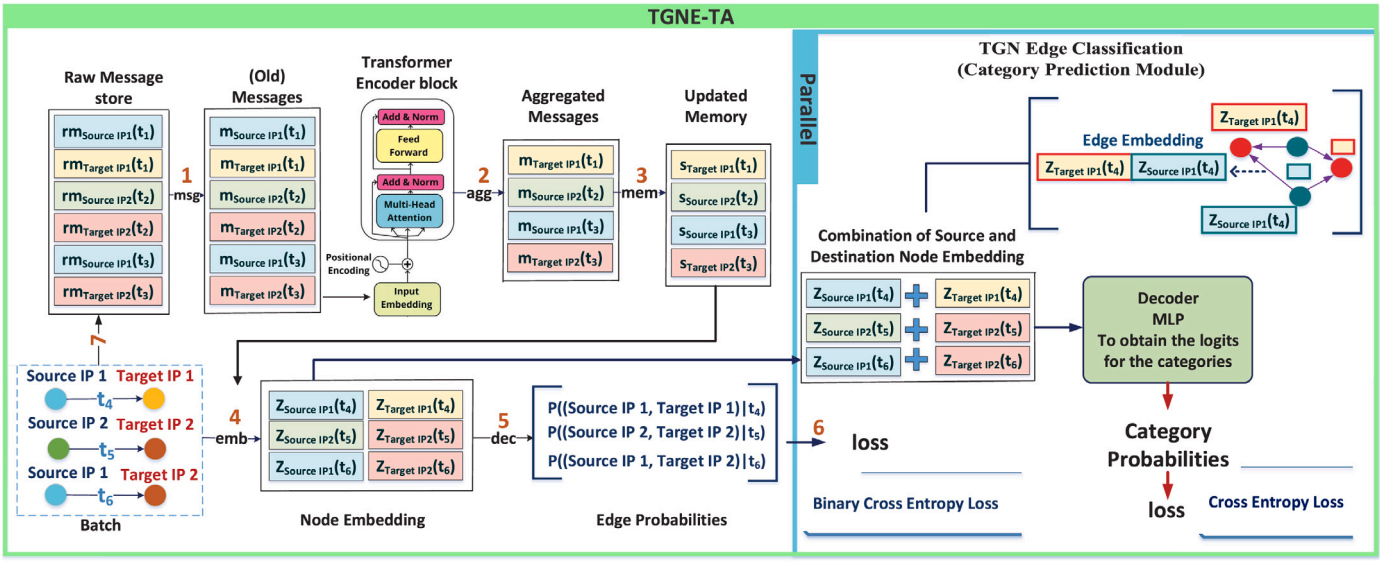


Fig. 6. Computations performed by TGNE-Transformer Aggregator on a batch of time-stamped interactions.

3.4.2. Ablation on temporal encodings

While prior work has emphasized timestamp-based encodings, we observed that index-based positional encoding led to superior performance in our link prediction tasks. One plausible explanation is that for many real-world applications, the order of interactions matters more than the precise time between them. Furthermore, timestamp-based encodings may introduce noise due to uneven or bursty interaction patterns. Our ablation study supports this observation (see Experiments section, Table 9).

3.5. Algorithmic overview

This section outlines three key algorithms for temporal graph modeling: The TGN, TGNE and TGNE-TA. The summary of main notations is in Table 2.

Algorithm 1 Temporal Graph Network (TGN)

procedure TGN(G, T, X_v, X_e, L, H, n) ▷ Graph G with timestamps T, node features X_v, edge features X_e, layers L, heads H, neighborhood size n
  Initialize embeddings $Z_v = X_v$ for each node $v \in V$
  for each time t and edge (u, v) with timestamp t **do**
    Retrieve Historical Neighbors $\mathcal{N}_t(v)$ up to n nodes based on prior interactions
    Message Passing
    for each layer $l = 1$ to L **do**
      Aggregate messages from $\mathcal{N}_t(v)$ using attention mechanism with H heads
      Update Node Embeddings for each node v by applying the attention layer and aggregating embeddings
    end for
    Memory Update: Update each node's memory state based on interaction history
  end for
  Edge Probability Computation
  Concatenate embeddings Z_u and Z_v for each node pair (u, v)
  Compute Probability $P(u, v)$ using an MLP or link predictor
  return node embeddings Z and edge probabilities P
end procedure

Algorithm 2 TGN with Edge Classification

Input: Same as TGN, with additional category predictor parameters
Output: Temporal node embeddings Z, edge embeddings $Z_{u,v}$, edge probabilities P, edge categories C

1: Run steps 1-5 of TGN to obtain node embeddings Z and edge probabilities P.
2: **Category Prediction**:
3: **a. Combine Embeddings**: For each edge (u, v), compute $Z_{u,v} = Z_u + Z_v$.
4: **b. Multi-class Prediction**: Pass $Z_{u,v}$ through a category prediction layer (e.g., a linear layer with softmax) to obtain category logits $C(u, v)$.
5: **Loss Calculation**:
6: **a.** Compute edge classification loss from $P(u, v)$ and $C(u, v)$.
7: **Return** node embeddings Z, edge embeddings $Z_{u,v}$, edge probabilities P, and edge categories C.

Algorithm 1: Temporal Graph Network (TGN)

The Temporal Graph Network (TGN) computes temporal embeddings for nodes and edge probabilities, which represent the likelihood of edges forming between nodes at any given time. This algorithm operates as follows:

1. Input and Initialization: TGN takes as input a graph $G = (V, E)$ with nodes V, edges E, associated timestamps T, node features X_v, edge features X_e, along with key hyperparameters like the number of layers L, attention heads H, and neighborhood size n. Each node v is initialized with an embedding equal to its features, $Z_v^{(0)} = X_v$, to capture initial node information.

2. Temporal Message Passing: For each edge (u, v) with a specific timestamp t, TGN retrieves the historical neighborhood of v, denoted $\mathcal{N}_t(v)$, up to a specified size n. In each layer of the model, TGN aggregates messages from these neighbors using an attention mechanism with H heads, enhancing the ability to learn from interactions. This process iteratively updates the node embeddings through the network's layers, capturing the evolution of node states.

3. Memory Update: TGN optionally updates node memories, which store interaction history, to maintain long-term dependencies between nodes.

Table 2
Summary of main notations used in the proposed models: TGN (binary), TGNE (multi-class), and TGNE-TA (Transformer Aggregator).

Notation	Description
$G = (V, E)$	Dynamic bipartite graph with nodes V and temporal edges E
$v_i, v_j \in V$	Node identifiers (IP addresses), can be attackers or victims
$e = (v_i, v_j, t, \ell)$	Temporal edge from v_i to v_j at time t with label ℓ
T	Timestamp of the interaction
ℓ	Edge label (binary or multi-class attack type)
X_v	Node features
X_e	Edge features
h_i^t	Node memory or hidden state for node v_i at time t
m_i^t	Aggregated message for node v_i at time t
d_{msg}	Dimension of message vectors in $\mathbb{R}^{d_{\text{msg}}}$
d_{mem}	Dimension of node memory vector space $\mathbb{R}^{d_{\text{mem}}}$
d_{emb}	Dimension of node embeddings in $\mathbb{R}^{d_{\text{emb}}}$
n_{layers}	Number of layers in TGNE/TGNE-TA
n_{heads}	Number of attention heads in transformer aggregation
$n_{\text{neighbors}}$	Number of sampled temporal neighbors
ϕ_{msg}	Message function for computing $m_{k\to i}^t$
ϕ_{upd}	Memory-update function for computing h_i^t
ϕ_{emb}	Embedding-projection function for node embeddings
$\mathcal{N}_i^t$	Set of neighbor nodes interacting with v_i before time t
$\mathcal{A}(\cdot)$	Aggregator function (e.g., mean, last, transformer)
L	Number of transformer layers
H	Heads
n	Neighborhood size
Z	Temporal node embeddings
$Z_{u,v}$	Edge embeddings
P	Edge probabilities
C	Edge categories
TE(t)	Temporal encoding of timestamp t
PE(t)	Positional encoding for ordered sequences
y_{ij}^t	Ground-truth label for interaction between v_i and v_j at time t
$\hat{y}_{ij}^t$	Model prediction for the same interaction
$\mathcal{L}$	Loss function used for training (e.g., cross-entropy)

4. Edge Probability Computation: After embedding updates, TGN concatenates the embeddings of node pairs (u, v) to compute edge probabilities $P(u, v)$ using a multi-layer perceptron (MLP) or link predictor. This step assesses the likelihood of an edge existing between nodes based on their temporal and feature-based embeddings.

Algorithm 2: TGN with Edge Classification

Building upon TGN, this variant also predicts categories for edges to enhance edge-level understanding, which is valuable for classification tasks within dynamic graphs.

1. Running TGN Steps: The first steps mirror TGN, generating temporal node embeddings Z.

2. Category Prediction: For each edge (u, v), node embeddings Z_u and Z_v are combined (typically through addition) to create a joint embedding $Z_{u,v} = Z_u + Z_v$. This joint embedding passes through a classification layer (such as a softmax layer) to predict edge categories, denoted $C(u, v)$.

3. Loss Calculation: To improve model performance, the algorithm computes a combined loss that includes both edge classification loss and edge probability loss. This dual-objective optimization ensures that the model not only predicts the likelihood of edge formation but also categorizes the edge based on learned embeddings. The time complexities are detailed in Appendix A.

3.6. Mathematical formulation of TGNE

Let $G = (V, E)$ be a dynamic bipartite graph where each temporal edge $e = (v_i, v_j, t, \ell) \in E$ denotes an interaction between nodes v_i and v_j at timestamp t with category $\ell \in \{1, \ldots, C\}$. Let X_v and X_e denote node and edge features, respectively. Each node v_i maintains a time-dependent memory state $h_i^t \in \mathbb{R}^{d_{\text{mem}}}$, where d_{mem} is the dimensionality of the memory vector.

Initialization: Each node $v \in V$ is associated with a raw feature vector X_v, and each edge $e \in E$ is associated with a raw feature vector X_e. The initial node and edge embeddings are defined as

$$Z_v^{(0)} = X_v, \qquad Z_e^{(0)} = X_e.$$

If the memory module is enabled, each node initializes a memory state

$$h_v^0 \in \mathbb{R}^{d_{\text{mem}}},$$

where d_{mem} is the memory dimension.

The model also initializes the hyperparameters including the number of layers (n_{layers}), number of attention heads (n_{heads}), message dimension (d_{msg}), memory dimension (d_{mem}), embedding dimension (d_{emb}), and number of edge categories (C) for classification.

Temporal Embedding Computation: For a given edge interaction at time t between source and destination nodes i, j:

1. Neighbor Sampling
   Sample $n_{\text{neighbors}}$ neighbors for nodes i and j. Let $\mathcal{N}(i)$ and $\mathcal{N}(j)$ be the sets of sampled neighbors.
2. Message Passing
   Compute messages from neighbors based on their embeddings and edge features. Aggregate the messages to update the temporal embeddings for nodes i, j.
   for a node i:

   $$\mathbf{m}_i(t) = \textbf{Aggregate}\left(\left\{\textbf{Message}(\mathbf{h}_k(t), \mathbf{e}_{ki}(t)) \mid k \in \mathcal{N}(i)\right\}\right)$$

   where $\mathbf{h}_k(t)$ is the embedding of neighbor k at time t and $\mathbf{e}_{ki}(t)$ is the edge feature between k and i.
3. Embedding Update: Update the node embeddings using the aggregated messages.

   $$\mathbf{h}_i(t) = \textbf{Update}(\mathbf{h}_i(t - \Delta t), \mathbf{m}_i(t))$$

   where $\mathbf{h}_i(t - \Delta t)$ is the previous embedding of node i.

Edge Probability Computation: For each edge interaction (i, j) at time t:

1. Compute Temporal Embeddings

$$\mathbf{h}_i(t), \mathbf{h}_j(t) = \textbf{temporal_embeddings}(i, j, t)$$

2. Edge Score Calculation
Compute the scores for positive and negative edges using the embeddings.

$$s_{ij}^{\textbf{pos}} = \sigma(\mathbf{h}_i(t) \cdot \mathbf{h}_j(t))$$

$$s_{ij}^{\textbf{neg}} = \sigma(\mathbf{h}_i(t) \cdot \mathbf{h}_k(t)), \quad k \neq j$$

where σ is the sigmoid function.

Category Prediction: For each edge interaction (i, j):

1. Combine the embeddings of source and destination nodes.

$$\mathbf{h}_{ij} = \mathbf{h}_i(t) + \mathbf{h}_j(t)$$

2. Category Logits Calculation
Compute the logits for each category using a linear layer.

$$\mathbf{z}_{ij} = \mathbf{W}\mathbf{h}_{ij} + \mathbf{b}$$

where $\mathbf{W}$ and $\mathbf{b}$ are the weights and biases of the linear layer, respectively.

3. Category Probabilities
Apply the softmax function to obtain the category probabilities.

$$\mathbf{p}_{ij} = \textbf{softmax}(\mathbf{z}_{ij})$$

Loss Computation: The total loss for training the model is a combination of the binary cross-entropy loss for edge existence and the cross-entropy loss for edge category prediction:
Binary Cross-Entropy Loss for Edge Existence:

$$\mathcal{L}_{\textbf{edge}} = -\sum \left(y_{ij} \log s_{ij}^{\textbf{pos}} + (1 - y_{ij}) \log(1 - s_{ij}^{\textbf{pos}}) \right)$$

where y_{ij} is the ground truth label indicating the existence of the edge.
Cross-Entropy Loss for Edge Category Prediction:

$$\mathcal{L}_{\textbf{category}} = -\sum \left(\mathbf{y}_{ij} \cdot \log \mathbf{p}_{ij} \right)$$

where $\mathbf{y}_{ij}$ is the one-hot encoded ground truth category.
Total Loss:

$$\mathcal{L} = \mathcal{L}_{\textbf{edge}} + \alpha \mathcal{L}_{\textbf{category}}$$

where α is a hyperparameter to balance the two loss components.
Algorithm 3: TGNE with Transformer Aggregator (TGNE-TA)

This variant enhances the standard TGNE by introducing a Transformer-based aggregator to better capture expressive, temporal, and position-aware interactions among neighboring nodes.
1. Input and Initialization: The algorithm takes as input a dynamic graph $G = (V, E)$, a set of timestamps T, node features X_v, edge features X_e, number of Transformer layers L, number of heads H, and neighborhood size n. Each node v is initialized as:

$$Z_v^{(0)} = X_v$$

2. Temporal Message Passing with Transformer Aggregator: For each edge $(u, v, t) \in E$, retrieve the historical neighborhood $\mathcal{N}_t(v)$ of node v using a temporal neighbor finder, up to size n:

$$\mathcal{N}_t(v) = \text{NeighborFinder}(v, t, n)$$

For each neighbor $k \in \mathcal{N}_t(v)$, a message is computed using the neighbor's embedding $\mathbf{h}_k(t_k)$, the edge features $\mathbf{e}_{kv}(t_k)$, and a temporal encoding of the time difference $\Delta t = t - t_k$:

$$\mathbf{m}_k = \text{Message}\left(\mathbf{h}_k(t_k), \mathbf{e}_{kv}(t_k), \text{TimeEncode}(t - t_k)\right)$$

All such messages form a sequence:

$$\mathcal{M}_v = \left[\mathbf{m}_1, \mathbf{m}_2, \ldots, \mathbf{m}_n\right]$$

Temporal or Positional Encoding: To encode temporal order or distance, either *Temporal Encoding* or *Positional Encoding* is added:

$$\mathcal{M}_v' = \mathcal{M}_v + \text{Temporal Encoding}(\Delta t_1, \ldots, \Delta t_n)$$

Transformer Aggregation: The modified sequence is fed into a multi-head Transformer encoder:

$$\mathbf{h}_v(t) = \text{Transformer}\left(\mathcal{M}_v'\right)$$

If $L > 1$, this process is repeated L times to allow deeper interaction modeling.
3. Memory Update: If memory is used, the updated embedding $\mathbf{h}_v(t)$ is stored in memory for future computations, capturing long-term node dynamics.
4. Edge Prediction and Classification: Given embeddings $\mathbf{h}_i(t)$ and $\mathbf{h}_j(t)$ for an edge (i, j), the model performs:
- **Link Prediction:** Compute edge score using dot product or MLP:

$$s_{ij} = \sigma(\mathbf{h}_i(t) \cdot \mathbf{h}_j(t))$$

- **Edge Classification:** Combine embeddings and apply softmax classification:

$$\mathbf{z}_{ij} = \mathbf{W}(\mathbf{h}_i(t) + \mathbf{h}_j(t)) + \mathbf{b}, \quad \mathbf{p}_{ij} = \text{softmax}(\mathbf{z}_{ij})$$

5. Loss Computation: Final loss combines binary cross-entropy for link prediction and categorical cross-entropy for edge classification:

$$\mathcal{L} = \mathcal{L}_{\text{edge}} + \alpha \mathcal{L}_{\text{category}}$$

Mathematical Update to Aggregation Function: Unlike traditional aggregators (e.g., mean or sum), the Transformer replaces explicit aggregation with self-attention:

$$\mathbf{h}_i(t) = \text{Transformer}\left(\left[\mathbf{m}_k + \text{PE}(\Delta t_k)\right]_{k \in \mathcal{N}_t(i)}\right)$$

Where PE is the positional or temporal encoding injected to each message.

Remarks: This design enables the model to capture both the content and the order of historical events. Experimental results (Section Experiments) show that this architecture significantly improves both transductive and inductive performance compared to classical aggregators.

4. Experiments

Generally, there are two primary approaches to alert prediction: sequential algorithms and graph modeling. Thus, we examined both sequential methods and graph-based methods. This paper investigates eight deep neural networks for alert prediction, including the TGN-attention, TGN with Edge Classification capabilities (TGNE) and TGNE with transformer encoder aggregator (TGNE-TA). The methods compared with these proposed approaches include the Joint Dynamic User-Item Embedding Model (JODIE), Graph Convolutional Network (GCN), GraphSAGE, Gated Recurrent Unit (GRU), and Bidirectional Long Short-Term Memory (BiLSTM) networks. Among these, TGN-attention, TGNE, TGNE-TA, and JODIE are temporal approaches, while GCN and GraphSAGE are static methods. In contrast, GRU and BiLSTM are sequential, non-graph-based approaches. We conducted a total of 22 experiments, 3 of which are detailed in Appendices B and C.

Evaluation Metrics

We evaluate the Temporal Graph Network models using key metrics that capture link prediction quality, category prediction performance, and model behavior.
Global Link Prediction Metrics: We use Average Precision (AP), AUC-ROC (AUC), Mean Reciprocal Rank (MRR), and Hits@K to assess the model's ability to correctly rank positive edges above negative ones.
Classification Metrics: For models that also predict edge categories (TGNE, TGNE-TA), we report Accuracy, Recall, and F1-score to capture overall correctness and stability under class imbalance.

Algorithm 3 TGNE with Transformer Aggregator

Require: Graph $G = (V, E)$, timestamps T, node features X_v, edge features X_e, time encoder $\mathcal{T}$, Transformer aggregator $\mathcal{A}_{\text{Transformer}}$, number of layers L, neighborhood size n

Ensure: Temporal node embeddings $\mathbf{h}_v(t)$ and edge predictions

for each node $v \in V$ **do**
  Initialize embedding $\mathbf{h}_v(0) \leftarrow X_v$
end for
for each edge interaction $(u, v, t) \in E$ **do**
  Retrieve $\mathcal{N}_t(v) \leftarrow \text{NeighborFinder}(v, t, n)$
  for each neighbor $k \in \mathcal{N}_t(v)$ **do**
    Compute message:

$$\mathbf{m}_k \leftarrow \text{Message}(\mathbf{h}_k(t_k), \mathbf{e}_{kv}(t_k), \mathcal{T}(t - t_k))$$

  end for
  Build input sequence: $\mathcal{M}_v = [\mathbf{m}_k]_{k \in \mathcal{N}_t(v)}$
  Inject temporal encoding:

$$\mathcal{M}'_v \leftarrow \mathcal{M}_v + \text{TemporalEncoding}(\Delta t_k)$$

  Aggregate using Transformer:

$$\mathbf{h}_v(t) \leftarrow \mathcal{A}_{\text{Transformer}}(\mathcal{M}'_v)$$

end for
for each edge (i, j, t) **do**
  Compute score: $s_{ij} = \sigma(\mathbf{h}_i(t) \cdot \mathbf{h}_j(t))$
  Compute category logits: $\mathbf{z}_{ij} = \mathbf{W}(\mathbf{h}_i(t) + \mathbf{h}_j(t)) + \mathbf{b}$
  Compute probabilities: $\mathbf{p}_{ij} = \text{softmax}(\mathbf{z}_{ij})$
end for
Compute total loss:

$$\mathcal{L} = \mathcal{L}_{\text{edge}} + \alpha \mathcal{L}_{\text{category}}$$

Rate-based Metrics: False Positive Rate (FPR), False Negative Rate (FNR), True Positive Rate (TPR), and True Negative Rate (TNR) summarize the model's error tendencies and reveal potential bias toward positive or negative predictions.

Loss Functions: Edge loss (binary cross-entropy or ranking loss) measures link prediction quality, while category loss (multi-class classification loss) evaluates edge-type prediction. Joint optimization of these losses supports simultaneous link and category prediction.

Overall, this metric set enables a concise yet comprehensive evaluation of both prediction tasks, balancing ranking performance, classification behavior, and error analysis.

4.1. Implementation

All experiments were implemented in Python and executed in a cloud-based environment equipped with an NVIDIA Tesla T4 GPU, 2 vCPUs, and 16 GB of RAM. The runtime configuration and memory management were optimized to efficiently handle large-scale dynamic graph datasets. Data storage and access were managed through seamless integration with cloud-based file systems. The code is available.[2]

4.2. Experiment 1: TGN-attention evaluation

For the evaluation of the TGN-attention model for link prediction within an evolving attack graph, binary classification was applied to detect daily interactions between attacker and victim. Initial testing focused on individual days, progressing to multi-day analyses to capture temporal patterns and trends. Table 3 shows performance metrics, including AUC for distinguishing attack instances, AP for accuracy, and MRR as a ranking metric. These metrics were calculated for both transductive and inductive tasks, with inductive performance generally lower, as expected, given the challenges with unseen nodes. Temporal analysis revealed that metric values varied based on sample sizes, as illustrated by higher performance on March 11, which had more samples than March 12 (March 12 has 10× fewer samples than the surrounding days). When the dataset size collapses to this extent, metrics such as AUC, AP, and MRR become unreliable because the classifier receives far fewer positive and negative examples. This explains the performance drop in metrics on March 12 compared to March 11. It is worth noting that class imbalance in the Warden Alert dataset is handled via controlled over- and under-sampling. Further analysis in Table 3 evaluated the model on the original dataset. Combining data improved overall metrics, highlighting the benefits of larger datasets. For comparing the balanced dataset evaluation results, see Table 4.

4.3. Experiment 2: TGNE-attention evaluation

In extending TGN, we enhanced binary link prediction and incorporated attack type prediction. This improvement enables simultaneous prediction of both the next victim and the attack type. The comparison between the results from the original dataset in Table 5 and the balanced data in Table 6 show no significant difference. This suggests that the model is capable of effectively learning from the original dataset without requiring balancing techniques. Table 5 presents the results of the TGNE-attention model, evaluated using AUC, average precision, MRR, and attack type prediction accuracy. By comparing the results of the proposed method, we conclude that in addition to accurately predicting links, the model achieves a 99% accuracy in predicting the link type. The near-identical performance indicates that the class distribution in the unbalanced data was not severe enough to impact the results, also the model's architecture and features are robust enough to handle class unbalances. This finding implies that, for this particular task, balancing the dataset may not be critical to achieving high accuracy and reliable predictions.

4.4. Experiment 3: Model performance stability in time

The evaluation results for the proposed temporal embedding approach were obtained using alerts from March 11-17, 2019. Training and testing were conducted on different dates from this entire week. Additionally, variations in prediction accuracy over time were explored across 12 different scenarios, as detailed in Table 7. These results demonstrate that the model maintains acceptable performance across all scenarios. The model's accuracy improves with more training data, while limited data results in reduced prediction performance. Notably, Scenario 7 highlights that the system performs well even with significant gaps between the training and testing periods. In this case, the model was trained on data from March 11 and 12 but tested on March 16 and 17, yet still achieved satisfactory results. This suggests the model does not require frequent retraining, potentially needing updates only every few days to maintain effective performance.

4.5. Experiment 4: Attack prediction accuracy

The accuracy of each attack category was calculated across four daily attack types: Anomaly.Traffic, Availability.DDoS, Availability.DoS, and Recon.Scanning. After categorizing the interactions into inductive and transductive, some days were found to be missing certain attack types, with most single-day intervals notably lacking the Availability.DoS attack in the inductive category, as shown in Table 8. After analyzing daily accuracy for each category, we expanded the evaluation to include the entire week, allowing for a more comprehensive assessment of attack patterns. The TGNE-attention model consistently achieves over 95% accuracy in classifying most attack types across

[2] https://github.com/ZahraMakkiNayeri/TGNE-TA.git

Table 3
Temporal graph network-attention performance results.

	AUC		Average Precision		MRR	
Day	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive
11	0.9257	0.9704	0.9316	0.9721	0.9396	0.9822
12	0.5171	0.5692	0.5683	0.5753	0.7882	0.7709
13	0.8706	0.9509	0.8802	0.9444	0.9186	0.9604
14	0.7678	0.9322	0.7828	0.9321	0.9185	0.9634
15	0.8230	0.9312	0.8361	0. 9312	0.8969	0.9624
16	0.8984	0.9636	0.9125	0.9667	0.9071	0.9540
17	0.6424	0.8321	0.6689	0.8253	0.8365	0.9026
11–12	0.8782	0.9425	0.8894	0.9445	0.9451	0.9608
11 to 13	0.8622	0.9310	0.8602	0.9285	0.9258	0.9601
11 to 14	0.8532	0.9193	0.8621	0.9208	0.9360	0.9525
11 to 15	0.8521	0.9178	0.8665	0.9222	0.9163	0.9469
11 to 16	0.8654	0.9246	0.8852	0.9309	0.9222	0.9484
11 to 17	0.8936	0.9590	0.8956	0.9556	0.9164	0.9606

Table 4
Temporal Graph Neural Network-Attention(Balanced data) performance.

	AUC		Average Precision		MRR	
Day	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive
11	0.8816	0.9498	0.8873	0.9511	0.9230	0.9464
12	0.5688	0.5181	0.6234	0.6047	0.7919	0.7832
13	0.8234	0.9452	0.8182	0.9407	0.9075	0.9639
14	0.7959	0.9089	0.8115	0.9076	0.9531	0.9693
15	0.8133	0.9367	0.8194	0.9348	0.9051	0.9618
16	0.8844	0.9499	0.9026	0.9483	0.9514	0.9806
17	0.6285	0.7856	0.6316	0.7644	0.8522	0.8607
11–12	0.8536	0.9498	0.8738	0.9492	0.9408	0.9621
11 to 13	0.8494	0.9263	0.8540	0.9256	0.9371	0.9623
11 to 14	0.8487	0.9216	0.8561	0.9202	0.9366	0.9565
11 to 15	0.8427	0.9144	0.8618	0.9178	0.9330	0.9510
11 to 16	0.8875	0.9472	0.8980	0.9490	0.9258	0.9516
11 to 17	0.8795	0.9256	0.8891	0.9121	0.9280	0.9506

Table 5
Temporal graph network-attention Edge Classification performance results.

	AUC		Average Precision		MRR		Accuracy	
Day	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive
11	0.9307	0.9529	0.9235	0.9452	0.9371	0.9360	0.9506	0.9684
12	0.6340	0.6560	0.6171	0.5674	0.7162	0.6536	0.4090	0.2569
13	0.8386	0.9469	0.8139	0.9474	0.8826	0.9635	0.9759	0.9751
14	0.7356	0.9145	0.7402	0.9099	0.8563	0.8510	0.8813	0.9351
15	0.8381	0.9262	0.8702	0.9269	0.9318	0.9468	0.9792	0.9893
16	0.5359	0.9618	0.5271	0.9644	0.8461	0.9685	0.9682	0.9848
17	0.6077	0.7527	0.6273	0.7591	0.7592	0.8739	0.9090	0.8470
11–12	0.7622	0.8735	0.7814	0.8939	0.7144	0.9173	0.9192	0.9662
11 to 13	0.8301	0.9104	0.8398	0.9137	0.9477	0.9371	0.9927	0.9696
11 to 14	0.7267	0.8739	0.7286	0.8825	0.8724	0.9185	0.9422	0.9530
11 to 15	0.7274	0.8888	0.7167	0.9047	0.8610	0.9384	0.9703	0.9878
11 to 16	0.8705	0.8950	0.8602	0.8720	0.9856	0.9579	0.9956	0.9830
11 to 17	0.8661	0.9251	0.8808	0.9286	0.9593	0.9394	0.9704	0.9806

Table 6
Temporal Graph Neural Network-Attention Edge Classification (Balanced data).

	AUC		Average Precision		MRR		Accuracy	
Day	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive
11	0.8373	0.9272	0.8430	0.9313	0.8924	0.9538	0.5737	0.8597
12	0.5888	0.5921	0.6148	0.6004	0.9400	0.7976	0.7600	0.5838
13	0.8443	0.9350	0.8105	0.9359	0.8796	0.9658	0.9768	0.9777
14	0.6787	0.7493	0.6567	0.7566	0.9080	0.9484	0.5183	0.6984
15	0.7234	0.8854	0.7046	0.8702	0.9039	0.9410	0.8306	0.8128
16	0.7911	0.9444	0.8165	0.9365	0.9123	0.9755	0.9589	0.9830
17	0.6637	0.8257	0.7013	0.7998	0.8536	0.8871	0.8902	0.9067
11–12	0.7889	0.9308	0.8180	0.9355	0.9144	0.9559	0.9969	0.9917
11 to 13	0.8240	0.9092	0.8326	0.9097	0.9309	0.9571	0.9390	0.9710
11 to 14	0.8369	0.9159	0.8351	0.9178	0.9097	0.9448	0.9882	0.9833
11 to 15	0.7677	0.8998	0.7938	0.9002	0.8973	0.9396	0.9927	0.9754
11 to 16	0.7557	0.9092	0.7369	0.9160	0.8728	0.9316	0.9941	0.9962
11 to 17	0.7521	0.8771	0.7399	0.8514	0.8201	0.9129	0.9523	0.9548

Table 7
TGNE Model performance stability in time.

Scenario	11	12	13	14	15	16	17	Inductive loss	Transductive loss	Inductive accuracy	Transductive accuracy
1	Train			Test				0.077	0.279	0.9805	0.9564
2	Train				Test			0.112	0.059	0.9514	0.9802
3	Train					Test		0.116	0.005	0.9620	0.9815
4		Train				Test		0.045	0.015	0.9837	0.9866
5		Train			Test			0.060	0.007	0.9651	0.9689
6			Train		Test			0.053	0.000	0.9754	0.9858
7	Train					Test		0.099	0.343	0.9801	0.9582
8	Train		Test					0.069	0.023	0.9879	0.9824
9			Train			Test		0.167	0.057	0.8875	0.9577
10					Train		Test	0.341	0.001	0.8362	0.8840
11			Train				Test	0.215	0.063	0.9601	0.9280
12	Train						Test	0.172	0.033	0.9670	0.9716

Table 8
Attack prediction accuracy of proposed TGNE-attention performance results.

	Anomaly.Traffic		Availability.DDos		Availability.Dos		Recon.Scaning	
Day	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive
11	1.0	0.9870	0.9863	0.9867	Missing	1.0	0.9306	0.9706
12	1.0	1.0	1.0	0.5625	Missing	0.0	0.0	0.0105
13	1.0	0.9916	1.0	0.9938	Missing	1.0	0.9468	0.9485
14	0.8266	0.9538	0.9750	0.9655	Missing	1.0	0.7248	0.8766
15	0.9666	0.9976	0.3333	0.8876	Missing	Missing	0.9914	0.9900
16	1.0	0.9954	Missing	0.9677	0.9600	0.9810	Missing	Missing
17	0.0	0.5312	0.5	0.25	Missing	0.0	1.0	0.9943
11–12	0.4047	0.3652	0.8490	0.8664	0.0	0.0	1.0	0.9993
11 to 13	0.9347	0.9700	1.0	1.0	Missing	0.0	0.9694	0.9783
11 to 14	0.9820	0.9749	0.9736	0.9945	0.0	0.0	0.9960	0.9978
11 to 15	0.9906	0.9866	1.0	0.9921	0.9473	0.8713	0.8695	0.9702
11 to 16	0.9920	0.9492	1.0	1.0	Missing	Missing	0.9873	0.9902
11 to 17	0.9097	0.9293	0.9668	0.9722	1.0	1.0	0.9974	0.9970

multiple days. Although it has never reached 100% accuracy for all attack types in all scenarios, it has perfectly classified some attack types on certain days. In inductive mode, the accuracy reaches up to 99% for specific attacks. As more data is added, the model's performance improves, maintaining accuracy above 90%, even when tested on smaller daily datasets, highlighting its robustness.

4.6. Experiment 5: The effect of memory updater on attack-type

Fig. 7 presents the attack type prediction accuracy results for both inductive and transductive tasks across one week when using either the GRU or RNN as the memory updater module in the TGNE-attention. With GRU, accuracy based on attack type frequency consistently ranges between 92% and 99%–100%. The RNN configuration achieves similar accuracy levels, though it requires a longer execution time compared to the GRU. This comparison highlights the efficiency advantage of GRU.

4.7. Experiment 6: The effect of memory updater on evaluation metrics

As illustrated in Fig. 8, we compared the AUC, AP, and MRR between the GRU and the RNN as memory updaters. The evaluation metrics indicate that the GRU demonstrates superior performance, making it a more promising option for memory updating in the TGNE-attention model for the link and attack type prediction task on the alert-sharing platform dataset.

4.8. Experiment 7: Comparative analysis of approaches

The comparison of AUC, AP, and MRR across eight approaches is shown in Fig. 9. The comparative results of TGN-attention, TGNE-attention, TGNE-TA, JODIE, GRU, BiLSTM, GCN, and GraphSAGE illustrate that TGN-attention, TGNE-attention and TGNE-TA deliver the highest performance, exceeding 90% across metrics. Proposed models effectively predict all attacker-victim interactions over time, increasing accuracy as the week progresses. TGN-attention achieves the top AUC and AP scores on March 17, while GraphSAGE achieves a perfect

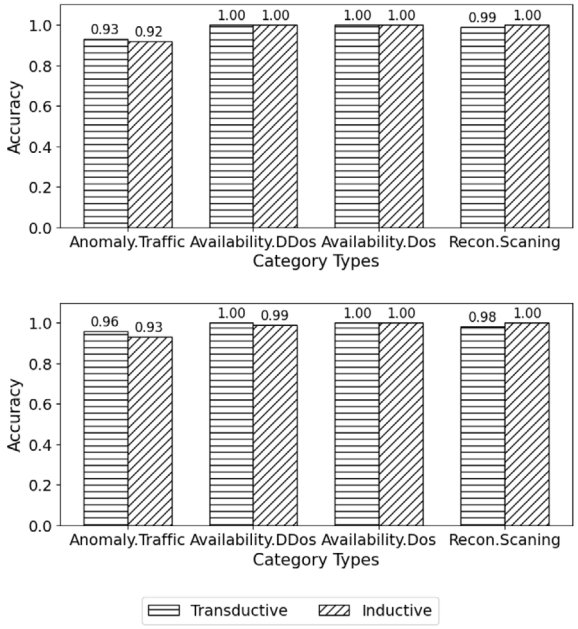


Fig. 7. Total accuracy for each attack category in a week. GRU memory updater (top), RNN memory update (bottom)(Inductive vs. Transductive).

MRR of 1 throughout, with TGN-attention as a close second. TGNE-attention and TGNE-TA closely parallel TGN-attention's performance, additionally predicting attack types.

4.9. Experiment 8: Accuracy comparative analysis

Fig. 10 shows that TGNE-attention achieves the highest accuracy among JODIE, GRU, BiLSTM, GCN, and GraphSAGE, proving its effectiveness in predicting attacker-victim interactions. Graph-based models,

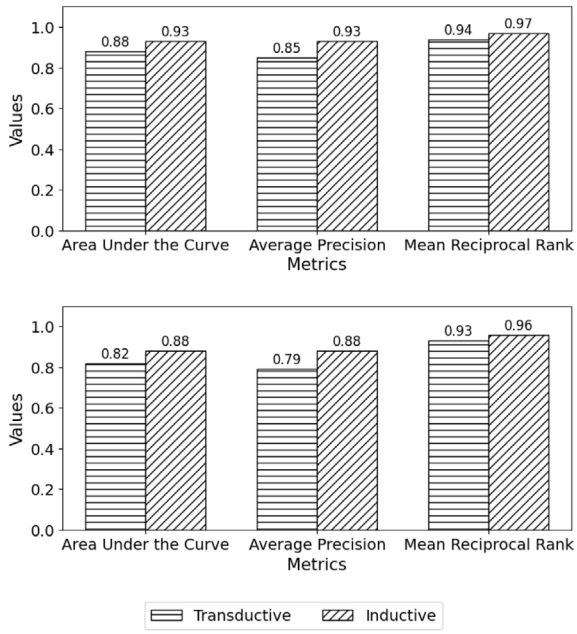


Fig. 8. Inductive vs. Transductive metric comparison in a week. GRU memory updater (top), RNN memory update (bottom).

particularly TGNE-attention, GraphSAGE, and GCN, excel in capturing complex network structures within alert data. While BiLSTM and GRU perform well in sequential tasks, they fall short here due to limitations in modeling graph dependencies. JODIE, a dynamic embedding model, has the lowest accuracy, indicating that while it handles temporal aspects, it may miss complex attack patterns. TGNE-attention's high accuracy emphasizes the value of attention in temporal graph networks for precise alert prediction.

4.10. Experiment 9: Error evaluation for each attack category

As shown in Fig. 11, model loss decreases across attack types in the category prediction task. when the daily data volume increases. Although the inductive loss sometimes appears lower than the transductive loss, this does not indicate stronger generalization. Instead, the inductive evaluation set is substantially smaller (10% of the data) and often lacks rare categories such as Availability.DoS on specific days (see Table 8). Missing or sparse categories reduce measured loss by providing fewer difficult examples, artificially lowering inductive loss despite the model having less information. This explains why inductive loss may appear better on some days.

4.11. Experiment 10: Encoding comparison

In our ablation study (see Table 9), we compare Temporal Encoding and Positional Encoding strategies in the Transformer-based aggregator. Surprisingly, positional encoding performs slightly better overall, including in the inductive setting. This suggests that in our temporal graph scenario, relative position-based representations are sufficient for capturing temporal dependencies and structural signals—potentially due to the regularity of interaction patterns and the short-range nature of most temporal dependencies.

The performance boost obtained by combining both encodings indicates that each captures complementary aspects of time. While Time Encode handles micro-level dynamics between interactions, the Temporal Encoding and Positional encoding helps the aggregator model macro-level temporal sequences. Although temporal encoding is designed to explicitly capture time gaps between interactions, our findings show that positional encoding can outperform it in certain scenarios—possibly due to better alignment with the attention mechanism's inductive biases.

4.12. Experiments 11: Effect of Dual-Level Time Encoding (DLTE)

To assess the contribution of our proposed Dual-Level Time Encoding (DLTE) component, we conducted an ablation study comparing TGNE-TA with and without this temporal encoding mechanism under both transductive and inductive settings. DLTE integrates both local interaction-level and global sequence-level temporal signals into the Transformer Aggregator, enabling the model to better capture temporal structure and timing in dynamic graphs. As reported in Table 10, incorporating DLTE consistently improves performance across all evaluated metrics, in both transductive and inductive scenarios. This improvement confirms that temporal modeling at multiple granularities plays a critical role in enabling the model to generalize to future interactions and to preserve temporal coherence across message sequences. The most substantial gains are observed in metrics related to ranking and recall, such as MRR and Hits@K, suggesting that DLTE enhances the model's ability to assign higher relevance to correct targets in both familiar and novel temporal contexts. These findings validate the decision to adopt DLTE as a core component in the TGNE-TA architecture, and all subsequent results, related to TGNE-TA, reported in this paper are based on the full version of TGNE-TA with DLTE enabled.

4.13. Experiments 12: Daily performance analysis

To assess the temporal consistency and robustness of the proposed model, we conducted a daily evaluation over a one-week period, from March 11 to March 17, under both **transductive** (interactions between seen nodes) and **inductive** (interactions involving unseen nodes) settings. Tables 11 and 12 report the averaged daily results using the TGNE-Transformer Aggregator (TGNE-TA).

- **Main Metrics:** The model achieves strong and stable performance throughout the week across all metrics. In the transductive setting, it obtains an **AUC of 0.98**, **AP of 0.97**, and an **F1 score of 0.93**, indicating accurate detection and classification of warnings. The high **MRR (0.9842)** and **Hits@1 (0.9685)** scores reflect precise ranking capability, ensuring that relevant interactions are prioritized effectively.
  In the inductive case, despite the inherent challenge of generalizing to unseen nodes, the model maintains a comparable performance with only a slight drop: **AUC = 0.92**, **F1 = 0.82**, and **MRR = 0.9501**. These results confirm the model's ability to transfer learned temporal and structural patterns to new regions of the graph.
- **Auxiliary Metrics:** In terms of binary classification metrics, the model exhibits high **TPR (95.2% transductive/94.4% inductive)** and acceptable **TNR (92.0% / 95.3%)**, showing reliable detection of positive interactions with some tolerance for false positives. The **FPR and FNR** values remain low, especially in the transductive setting (**FPR = 7.91%, FNR = 4.77%**), which is crucial for applications involving early warning or fraud detection where minimizing false negatives is critical.
- **Top-K Ranking:** Both settings yield a perfect score of **Hits@3 and Hits@5 = 1.0**, indicating that the relevant warning interactions are always retrieved within the top-5 predictions, an essential feature in real-time response systems.

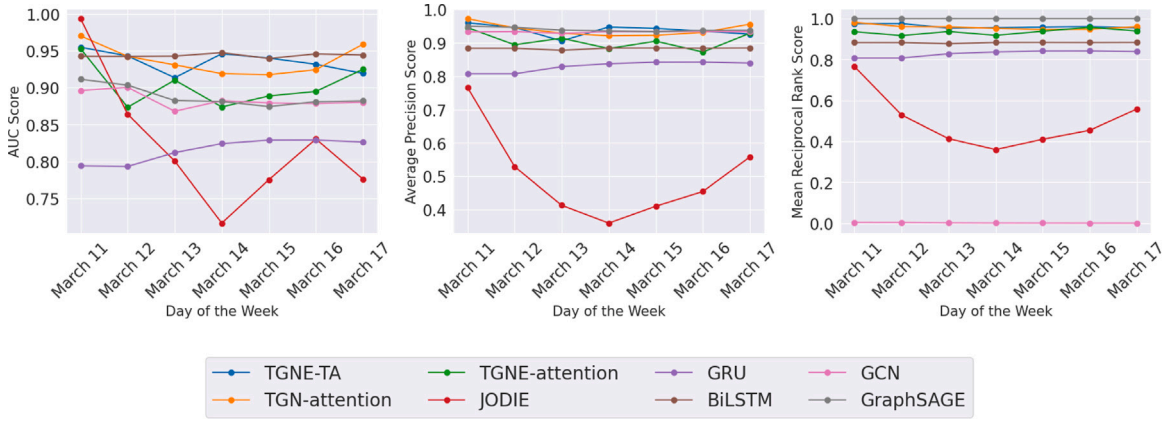


Fig. 9. Comparison of AUC, Average Precision, and Mean Reciprocal Rank for multiple approaches over time.

Table 9
Performance comparison between Temporal and Positional encoding within the Transformer aggregator, evaluated on old nodes (transductive setting) and new nodes (inductive setting) for a week.

Metric	Transductive (Temporal)	Inductive (Temporal)	Transductive (Positional)	Inductive (Positional)
AUC	0.9327	0.8973	0.9329	0.8991
AP	0.9319	0.9096	0.9309	0.9079
MRR	0.9599	0.9359	0.9631	0.9408
Recall	0.8727	0.8039	0.8800	0.8489
F1	0.8543	0.8138	0.8609	0.8177
FPR	0.1706	0.1719	0.1644	0.2272
FNR	0.1273	0.1960	0.1199	0.1510
TPR	0.8727	0.8040	0.8800	0.8489
TNR	0.8294	0.8281	0.8355	0.7727
Hits@1	0.9197	0.8719	0.9263	0.8816
Hits@3	1.0000	1.0000	1.0000	1.0000
Hits@5	1.0000	1.0000	1.0000	1.0000

Table 10
Performance Comparison of TGNE-TA with and without Dual-Level Time Encoding (DLTE) under Transductive and Inductive Settings.

Model	Setting	F1	AUC	AP	MRR	Recall	Hit@1	FPR	FNR
TGNE-TA	Transductive	0.8531	0.9242	0.9108	0.9529	0.8363	0.9058	0.1243	0.1636
TGNE-TA + DLTE	Transductive	**0.8609**	**0.9329**	**0.9309**	**0.9631**	**0.8800**	**0.9263**	0.1644	**0.1199**
TGNE-TA	Inductive	0.8121	0.8874	0.8923	0.9306	0.8134	0.8612	0.1896	0.1865
TGNE-TA + DLTE	Inductive	**0.8177**	**0.8991**	**0.9079**	**0.9408**	**0.8489**	**0.8816**	0.2272	**0.1510**

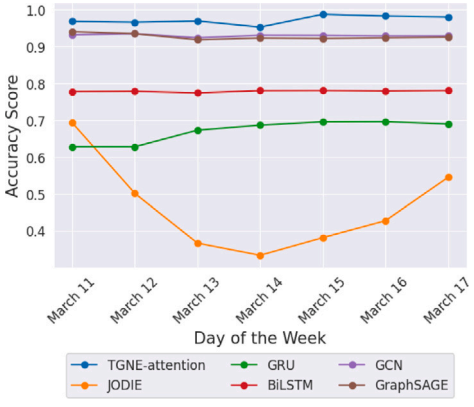


Fig. 10. Comparison of Accuracy for multiple approaches over time.

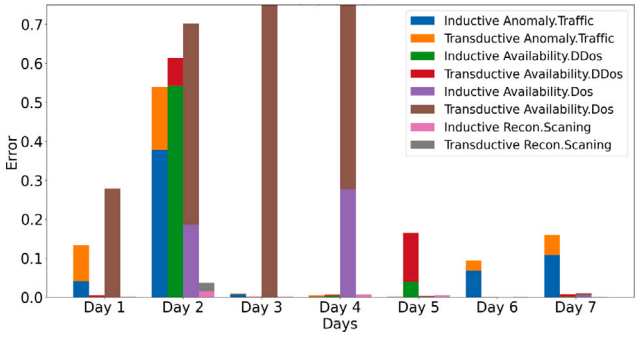


Fig. 11. Total error for each attack category by day (Inductive vs. Transductive).

Table 11
Daily evaluation results from March 11 to March 17 for both transductive (old nodes) and inductive (new nodes) settings based on TGNE-TA.

Date	Transductive (Old Nodes)						Inductive (New Nodes)					
	AUC	AP	F1	MRR	Recall	Hits@1	AUC	AP	F1	MRR	Recall	Hits@1
Mar 11	0.9549	0.9600	0.8891	0.9758	0.9048	0.9516	0.8935	0.9032	0.8020	0.9432	0.8261	0.8864
Mar 12	0.4863	0.4875	0.6666	0.7737	1.0	0.5474	0.4939	0.4992	0.6682	0.7714	1.0	0.5428
Mar 13	0.9621	0.9585	0.9014	0.9707	0.9451	0.9414	0.8954	0.8927	0.8246	0.9329	0.8577	0.8658
Mar 14	0.9228	0.9221	0.8645	0.9604	0.9182	0.9208	0.8089	0.8188	0.7601	0.9224	0.8818	0.8449
Mar 15	0.9175	0.8941	0.8953	0.9683	0.9418	0.9367	0.8411	0.8214	0.7888	0.9219	0.8947	0.8439
Mar 16	0.9799	0.9789	0.9375	0.9842	0.9522	0.9685	0.9239	0.9256	0.8445	0.9418	0.9202	0.8837
Mar 17	0.8231	0.8231	0.7896	0.9247	0.9462	0.8494	0.6225	0.6680	0.6709	0.8331	0.9448	0.6662
Mar 11–12	0.9431	0.9456	0.8627	0.9762	0.9191	0.9524	0.9012	0.9169	0.8094	0.9489	0.7115	0.8979
Mar 11–13	0.9137	0.9056	0.8638	0.9538	0.9088	0.9077	0.8783	0.8852	0.8007	0.9501	0.9094	0.9002
Mar 11–14	0.9461	0.9472	0.8664	0.9561	0.8572	0.9123	0.8758	0.8799	0.7913	0.9321	0.8740	0.8643
Mar 11–15	0.9402	0.9429	0.8487	0.9588	0.8764	0.9177	0.8735	0.8828	0.7856	0.9218	0.8584	0.8437
Mar 11–16	0.9318	0.9346	0.8419	0.9612	0.9058	0.9224	0.8995	0.9107	0.8114	0.9364	0.7748	0.8728
Mar 11–17	0.9196	0.9258	0.8399	0.9553	0.9011	0.9107	0.8811	0.8993	0.7963	0.9265	0.8645	0.8530

Table 12
Auxiliary daily evaluation metrics for March 11–17, based on TGNE-TA.

Date	Transductive (Old Nodes)						Inductive (New Nodes)					
	TPR	TNR	FPR	FNR	Hits@3	Hits@5	TPR	TNR	FPR	FNR	Hits@3	Hits@5
Mar 11	0.9048	0.8694	0.1306	0.0952	1.0	1.0	0.8261	0.7658	0.2342	0.1739	1.0	1.0
Mar 12	1.0	0.0	1.0	0.0	1.0	1.0	1.0	0.0071	1.0	0.0	1.0	1.0
Mar 13	0.9451	0.8480	0.1519	0.0548	1.0	1.0	0.8577	0.7773	0.2226	0.1422	1.0	1.0
Mar 14	0.9182	0.7938	0.2061	0.0817	1.0	1.0	0.8818	0.5615	0.4384	0.1181	1.0	1.0
Mar 15	0.9418	0.8379	0.1620	0.0581	1.0	1.0	0.8947	0.6261	0.3738	0.1052	1.0	1.0
Mar 16	0.9522	0.9208	0.0791	0.0477	1.0	1.0	0.9202	0.7410	0.2589	0.0797	1.0	1.0
Mar 17	0.9462	0.5495	0.4504	0.0537	1.0	1.0	0.9448	0.1282	0.8717	0.0551	1.0	1.0
Mar 11–12	0.9191	0.7882	0.2117	0.0808	1.0	1.0	0.7115	0.9534	0.0465	0.2884	1.0	1.0
Mar 11–13	0.9088	0.8046	0.1953	0.0911	1.0	1.0	0.9094	0.6379	0.3620	0.0905	1.0	1.0
Mar 11–14	0.8572	0.8783	0.1216	0.1427	1.0	1.0	0.8740	0.6651	0.3348	0.1259	1.0	1.0
Mar 11–15	0.8764	0.8111	0.1888	0.1235	1.0	1.0	0.8584	0.6731	0.3268	0.1415	1.0	1.0
Mar 11–16	0.9058	0.7540	0.2459	0.0941	1.0	1.0	0.7748	0.8650	0.1349	0.2251	1.0	1.0
Mar 11–17	0.9011	0.7553	0.2446	0.0988	1.0	1.0	0.8645	0.6934	0.3065	0.1354	1.0	1.0

4.13.1. Summary and practical implications

Overall, the daily evaluations demonstrate not only high predictive accuracy but also temporal robustness across multiple days. The model's consistent performance in both transductive and inductive settings highlights its ability to adapt to evolving network structures without significant loss in precision or recall. Such characteristics are especially valuable in real-world dynamic graph applications, such as warning systems in cybersecurity or communication networks, where new entities frequently emerge and prediction reliability must persist over time. Moreover, the stable performance observed over extended temporal spans suggests that the proposed framework scales with increasing data volume.

4.14. Experiment 13: Temporal generalization evaluation

To evaluate the generalization capacity of the proposed TGNE-TA model across temporal shifts, we designed four realistic train-test scenarios simulating deployment over consecutive days. Each setting involves training the model on a specific time window and testing on interactions from future days. The configurations include: 11–13 → 14 Mar, 11–12 → 16–17 Mar, 11–16 → 17 Mar, and 12–14 → 15–17 Mar. These experiments were conducted under both transductive and inductive conditions to capture performance on previously observed and entirely new nodes, respectively.

As shown in Table 13, TGNE-TA consistently outperforms the TGNE across all metrics and scenarios. This improvement is evident for both inductive and transductive tasks, confirming that the Transformer-based aggregator and dual-level temporal encoding enhance the model's ability to capture temporal dependencies and adapt to unseen patterns over time. Notably, the 12–14 → 15–17 Mar setting yielded the best overall results, highlighting the model's robustness when sufficient short-term history is available and the test interactions lie in a well-structured temporal window. The consistently superior performance across all evaluation metrics in this scenario reflects the model's capacity to preserve temporal coherence and accurately model evolving graph structures, even in inductive conditions. These results confirm that TGNE-TA exhibits strong temporal generalization, which is a crucial property for real-world dynamic graph applications, such as network alert systems and predictive monitoring platforms.

4.15. Experiment 14: Performance of simple heuristic baselines

We include two heuristic victim prediction baselines: Most Frequent Victim and Repeat Last Victim. These serve as lower bounds for the victim prediction subtask. Our model, in addition to predicting interaction types and probabilities, also identifies victim entities. As illustrated in Table 14, to fairly compare with these baselines, we compute accuracy, F1, precision, and recall over the predicted victim IDs. While these baselines are limited, they provide insight into the difficulty of the prediction task. Our model significantly outperforms both Most Frequent Victim and Repeat Last Victim, indicating its ability to go beyond frequency and recency-based signals.

4.16. Experiment 15: ROC-AUC evaluation for inductive and transductive settings

To comprehensively evaluate the binary classification performance of the proposed model, we report the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) under both transductive and inductive settings. The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold levels, providing a robust view of the model's discriminative capability across different decision boundaries.

Table 13
Comparison of TGNE and TGNE-TA across 4 temporal train-test scenarios.

ID	Scenario (Train → Test)	Model	F1	AUC	AP	MRR	Recall	Hits@1	FPR	FNR
Transductive (Old Nodes)										
1	11–13 → 14 Mar	TGNE	0.8203	0.9097	0.9114	0.9387	0.8907	0.8775	0.2809	0.1092
1	11–13 → 14 Mar	TGNE-TA	0.8666	0.9424	0.9437	0.9576	0.8599	0.9152	0.1247	0.1400
2	11–12 → 16-17 Mar	TGNE	0.8666	0.9359	0.9372	0.9634	0.8933	0.9268	0.1683	0.1066
2	11–12 → 16-17 Mar	TGNE-TA	0.8674	0.9412	0.9418	0.9657	0.9104	0.9315	0.1888	0.0895
3	11-16 → 17 Mar	TGNE	0.8544	0.8989	0.9154	0.9479	0.8569	0.8958	0.1491	0.1430
3	11-16 → 17 Mar	TGNE-TA	0.8699	0.9435	0.9347	0.9707	0.8601	0.9415	0.1175	0.1398
4	12-14 → 15-17 Mar	TGNE	0.8641	0.9402	0.9436	0.9567	0.8418	0.9135	0.1067	0.1581
4	12-14 → 15-17 Mar	TGNE-TA	0.8910	0.9603	0.9621	0.9708	0.8977	0.9416	0.1175	0.1022
Inductive (New Nodes)										
1	11–13 → 14 Mar	TGNE	0.7684	0.8450	0.8551	0.9187	0.9285	0.8375	0.9285	0.0714
1	11–13 → 14 Mar	TGNE-TA	0.7956	0.8714	0.8791	0.9274	0.8713	0.8548	0.3189	0.1286
2	11–12 → 16-17 Mar	TGNE	0.8216	0.8960	0.9065	0.9474	0.8606	0.8948	0.2342	0.1393
2	11–12 → 16-17 Mar	TGNE-TA	0.8152	0.8953	0.9037	0.9482	0.8320	0.8964	0.2091	0.1679
3	11-16 → 17 Mar	TGNE	0.7437	0.8247	0.8586	0.9132	0.6880	0.8265	0.1622	0.3119
3	11-16 → 17 Mar	TGNE-TA	0.7926	0.8791	0.8856	0.9306	0.8318	0.8612	0.2670	0.1681
4	12-14 → 15-17 Mar	TGNE	0.7988	0.8795	0.8932	0.9291	0.8009	0.8582	0.2043	0.1990
4	12-14 → 15-17 Mar	TGNE-TA	0.8241	0.9107	0.9199	0.9473	0.8547	0.8947	0.2194	0.1452

Table 14
Performance of Simple Heuristic Baselines for a week.

Method	Accuracy	Precision	Recall	F1	AUC	MRR	Hit@5
Most Frequent Victim	0.0204	0.0008	0.0016	0.0008	0.5	0.0205	1.0
Repeat Last Victim	0.0209	0.0010	0.0017	0.0011	0.5	0.0208	1.0

4.16.1. Transductive vs. Inductive performance

As shown in Fig. 12, the model achieves an ROC-AUC score of **0.93** in the transductive setting and **0.94** in the inductive setting. These values indicate strong classification ability, with only a minor drop in inductive performance, suggesting that the model generalizes well even to previously unseen nodes or interactions.

- **Transductive (AUC = 0.93):** In this setting, all nodes appear during training, allowing the model to fully leverage historical memory updates. The high AUC score reflects strong alignment between predicted probabilities and true labels, with minimal misclassification across thresholds.
- **Inductive (AUC = 0.94):** Here, the model must predict on interactions involving previously unseen nodes. Despite this added difficulty, the model maintains near-equivalent performance, highlighting its ability to transfer learned temporal and structural patterns to novel graph segments.

4.16.2. True Positive Rate (TPR) and True Negative Rate (TNR)

Alongside AUC scores, we report the TPR and TNR metrics to better capture the sensitivity and specificity of the model. Both metrics exceed **0.94**, underscoring balanced behavior:

- **TPR ≈ 0.94:** The model successfully captures the majority of warning (positive) instances, ensuring timely and reliable alerts in dynamic environments.
- **TNR ≈ 0.94:** The model effectively filters out benign interactions, maintaining a low false alarm rate — a critical feature for operational efficiency in real-world systems.

4.16.3. Analysis and implications

The near-parity in AUC and TPR/TNR across both evaluation settings confirms that the proposed model, enhanced with dual-level temporal encoding and transformer-based aggregation, maintains high fidelity in distinguishing between warning and non-warning interactions across dynamic graph topologies. This robustness is particularly valuable in real-time security and monitoring applications, where maintaining high true positive coverage while minimizing false positives is essential. The ROC curves also show steep rises in TPR with low FPR, confirming that the model remains confident and precise across a wide range of threshold values.

4.17. Experiment 16: Precision-recall analysis

In addition to ROC-AUC, we evaluate the model's performance using Precision-Recall (PR) curves under both transductive and inductive settings. While ROC-AUC captures the overall separability of classes, PR curves provide a more sensitive evaluation in imbalanced classification tasks such as warning prediction by emphasizing the model's precision in correctly identifying positive instances. Fig. 13 shows the PR curves for both settings. The proposed TGNE-Transformer model achieves a high area under the PR curve (PR-AUC) of **0.93** in the transductive setting and **0.94** in the inductive setting. These results indicate a strong ability to retain precision even at high recall levels.

4.17.1. F1 score comparison

To further quantify the balance between precision and recall, we report the F1 score, the harmonic mean of the two:

- **Transductive F1 = 0.85:** The model exhibits strong, balanced performance when predicting interactions between previously seen nodes. The high F1 reflects consistent precision and recall throughout the evaluation range.
- **Inductive F1 = 0.86:** Despite the challenge of generalizing to unseen nodes, the inductive setting results in slightly higher F1 and PR-AUC. This suggests that the model effectively captures transferable temporal and structural patterns that generalize beyond the training graph.

4.17.2. Interpretation and implications

The elevated PR-AUC and F1 scores in both evaluation settings highlight the model's robustness in high-recall regimes, where false positives often increase. Notably, the slightly higher scores in the inductive case suggest that the model's temporal encoding and Transformer-based aggregation promote generalization without overfitting to specific node histories. This characteristic is particularly valuable in practical scenarios, such as anomaly or warning detection in evolving network systems, where unseen entities frequently appear, and early yet accurate predictions are essential.

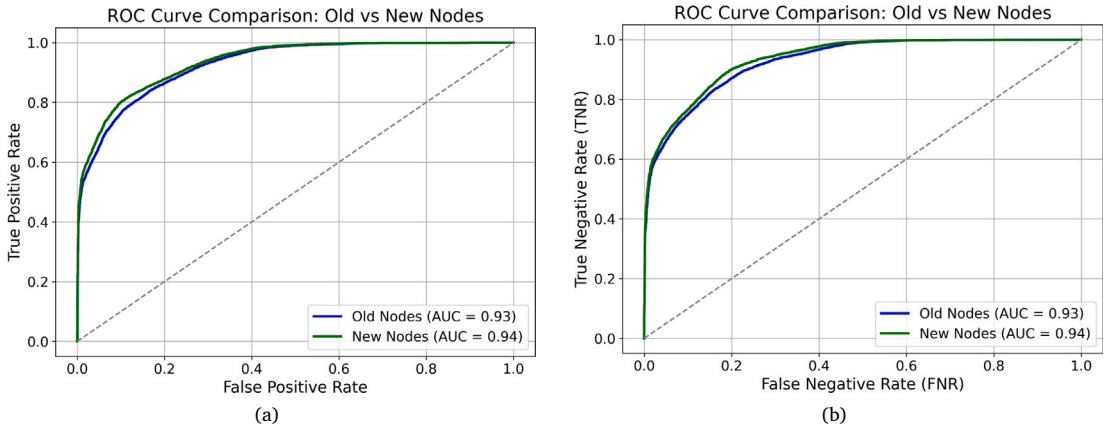


Fig. 12. TGNE-Transformer Aggregator ROC Curve Comparison. (a) TPR vs. FPR (b) TNR vs. FNR. The AUC of new nodes (inductive setting) is better than the AUC of old nodes (transductive setting).

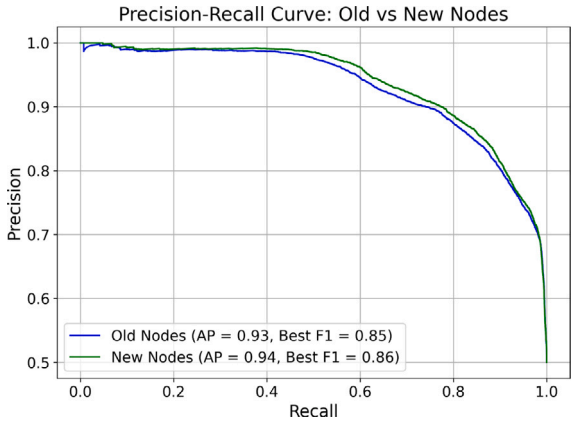


Fig. 13. Precision-Recall curve (Inductive vs. Transductive) with TGNE-Transformer Aggregator.

4.18. Experiment 17: Temporal horizons

To assess the temporal behavior of the proposed model in predicting warnings over dynamic interactions, we divided the test set into five temporal bins based on the relative position of each interaction within its sequence: Very Early, Early, Middle, Late, and Very Late. These bins capture different phases of interaction lifecycles, enabling fine-grained temporal analysis. Fig. 14 presents a box plot illustrating the distribution of predicted scores across these temporal bins. Each box summarizes the model's output confidence for interactions within a given temporal segment. This analysis effectively evaluates the model's temporal generalization across different prediction horizons.

4.18.1. Positive samples

Very Early & Early: The predicted scores in the early bins show moderate variance and relatively high median values. This suggests that the model can confidently identify warning-related interactions even at the early stages of node activity, likely due to effective encoding of recent memory updates and interaction context.

Middle: The middle bin exhibits a slightly narrower interquartile range, indicating more consistent predictions across samples. The median remains relatively stable, reflecting robust performance in the mid-sequence, where the model benefits from both past context and sufficient sequence information.

Late & Very Late: The scores in the later bins (especially Very Late) maintain a similar median compared to earlier bins but show a slightly increased spread (higher variance and presence of outliers). This behavior may be attributed to accumulated noise or less frequent pattern recurrence in later interactions, yet the absence of a sharp decline in performance demonstrates that the model generalizes well across temporal horizons.

4.18.2. Negative samples

To further evaluate the model's temporal consistency, we also analyzed the predicted scores for interactions labeled as negative. Across all bins, the predicted scores for negative samples are consistently low, with medians close to the minimum threshold. This indicates that the model reliably suppresses false positives regardless of when the interaction occurs in the sequence. The narrow interquartile ranges and dense lower-bound clustering reflect high confidence in negative predictions over time. Although a few outliers appear in each bin — particularly in the Very Late segment — these are relatively sparse and do not significantly affect the overall distribution. This stability reinforces the model's robustness not only in detecting true positives but also in confidently rejecting irrelevant or benign interactions throughout the temporal span. Overall, the temporal box plot analysis confirms that the model maintains consistent confidence for both positive and negative predictions across time, underscoring the effectiveness of the dual-level temporal encoding in capturing both short-term and long-term dynamics. Such robustness is particularly crucial in dynamic environments like network warning systems, where interactions can occur with irregular timing and patterns. The model's ability to preserve prediction confidence from Very Early to Very Late phases highlights its suitability for real-world temporal classification tasks.

4.19. Experiment 18: Effect of temporal distance on model predictions

To better understand how the temporal aspect influences the model's behavior, we analyzed the predicted scores of positive interactions as a function of their temporal distance. Fig. 15 illustrates this relationship, where the x-axis (log-scaled) represents the temporal distance (in seconds) between interactions, and the y-axis shows the predicted score produced by the model. For short-term interactions with a temporal distance below 10^2 s, the model assigns relatively high prediction scores, frequently exceeding 0.65. This indicates strong confidence when predicting near-future interactions, consistent with temporal locality often observed in dynamic graphs. In contrast, as the temporal distance increases (beyond 10^4 s), the predicted scores tend to decline and become more dispersed. This reflects increased uncertainty, potentially due to the reduced availability of long-range temporal patterns in the training data. The overall trend line shows a clear downward slope, suggesting that the model captures a temporal decay pattern: interactions that

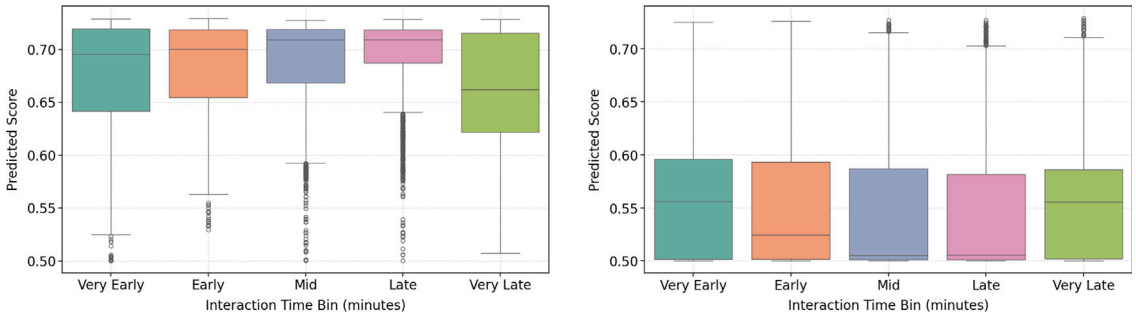


Fig. 14. TGNE-TA prediction score distribution across temporal bins. (a) Positive Labels (b) Negative Labels.

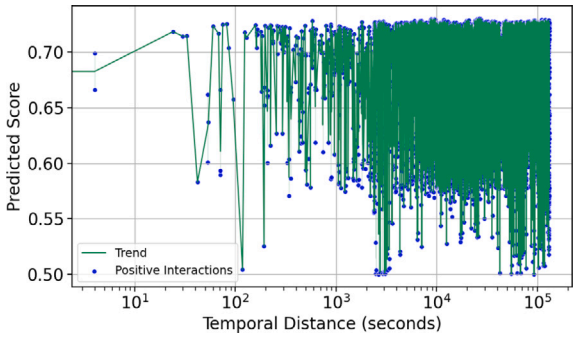


Fig. 15. Predicted score vs temporal distance (log scale).

Table 15
NF-UNSW-NB15-v2 Dataset-Evaluation metrics.

Metric	Inductive	Transductive
MRR	0.9344	0.9613
Recall	0.9845	0.9829
F1 Score	0.8534	0.9026
FPR	0.3227	0.1949
FNR	0.0154	0.0170
TPR	0.9845	0.9829
TNR	0.6772	0.8050
Hits@1	0.8688	0.9227
Hits@3	1.0	1.0
Category Accuracy	0.9248	0.8471

are temporally distant are considered less likely. Between 10^2 and 10^4 s, we observe substantial fluctuation in the predicted scores, pointing to possible instability in mid-range temporal predictions, which could stem from uneven sample distribution or behavioral complexity. These findings highlight the importance of fine-grained temporal modeling in dynamic graphs and support future work on incorporating explicit time-aware mechanisms to enhance prediction robustness.

4.20. Experiment 19: UNSW dataset

For evaluating the performance of our temporal graph-based model, we used the NF-UNSW-NB15-v2 dataset, which is a well-known benchmark for network intrusion detection tasks. This dataset contains traffic records labeled into ten distinct attack categories: *Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms* and *Normal*. The class distribution is highly imbalanced, with the Generic and Exploits classes dominating the dataset, while classes like Shellcode, Worms, and Backdoors are severely underrepresented. To address this class imbalance issue without altering the structure of the temporal dependencies, we applied a class-weighting strategy during training. This method assigns higher weights to minority classes in the loss function, allowing the model to learn more effectively from rare class samples while preserving the natural time-order and graph structure of the data. The results, summarized in Table 15, demonstrate consistent performance across a diverse range of evaluation criteria.

5. Related works

In cybersecurity forecasting for alert prediction, research generally falls into three categories: discrete models, continuous models, machine learning and data mining approaches (Husák et al., 2018). Discrete Models: Discrete models utilize defined states to predict specific attack scenarios. Graphical models like attack graphs, Bayesian networks, and Markov models are fundamental. Attack graphs, introduced by Hughes and Sheyner (2003), provide visual pathways for attack projection and intrusion recognition. Implementations such as the NICE framework by Chung et al. (2013) support countermeasure selection, while CAMIAC by Kotenko and Chechulin (2013) evaluates attack impact through graph-based approaches. Bayesian networks, often integrating attack graphs, provide real-time prediction improvements, as shown by Ramaki et al. (2015), who achieved higher accuracy with their Bayesian-based correlation framework. Okutan et al. (2017b) extended this approach by incorporating social media data, achieving prediction accuracies from 63% to 99%. Markov models, such as hidden Markov models, further aid multi-step attack prediction, as demonstrated by Sendi et al. (2012) in preventing control extension within networks. Continuous Models: Continuous models, particularly those using time series analysis, allow for real-time monitoring of security status changes. Time series models provide robust trend forecasting in network security. For example, Zhan et al. (2013) achieved 87.9% accuracy in predicting attacks one hour ahead, while Werner et al. (2017) and Sokol and Gajdoš (2018) used ARIMA and AR(1) models to anticipate attacks, showcasing time series' versatility in security forecasting. Machine Learning and Data Mining Approaches: Machine learning techniques, including neural networks and support vector machines, excel in recognizing patterns from historical data (Zhang and Shen, 2005). Data mining techniques further enhance predictive accuracy by extracting meaningful features (Husák et al., 2020, 2021). Recently, deep learning models have advanced alert prediction, Ansari et al. (2022), Cil et al. (2021), Buyuktanir et al. (2025), Büyüktanir et al. (2025), Anli et al. (2024), Zhao et al. (2024), Wang et al. (2023). Ansari et al. (2022) used GRU models to achieve over 76% accuracy, and Bartos et al. (2019) applied shallow neural networks for attack estimation. Shen et al. (2018) contributed with an RNN-based approach for sequential event prediction, underscoring the promise of deep learning in forecasting sequential security incidents. The comparisons (Ansari et al., 2020; Zhang et al., 2019; Husák et al., 2017; Ben Fredj et al., 2020; Lei and Li, 2007; Li et al., 2007; Hernández et al., 2016) is shown in Tables 16 and 17. Most existing works address individual aspects such as alert prediction, attack-type classification,

or attack graph construction in isolation, without jointly modeling attacker–victim interactions and attack types over time. In contrast, our approach advances beyond these methods by leveraging a dynamic temporal graph to enable unified predictions.

Case studies

Beyond attack forecasting, several studies provide valuable insights into network behavior, security assessment, and traffic analysis across different domains.

Anomaly Detection: Nia et al. (2016) proposed a pattern-based random-walk framework for detecting abnormal network traffic. The method constructs dynamic traffic patterns via a self-avoiding walk algorithm and adapts them in real time. Patterns are evaluated against a threat database using threshold-based conditions to identify malicious behavior.

AI in Cybersecurity: Babulak (2023) provides a comprehensive overview of modern AI-based methods for detecting, preventing, and mitigating sophisticated cyberattacks. The book surveys a wide range of AI tools — including machine learning, deep learning, and intelligent automation — and discusses their practical deployment in evolving threat environments.

QoS in Mobile Networks: Massam and Babulak (2003) focus on improving service quality in mobile communication networks by emphasizing the role of meaningful and enforceable service level agreements. With the emergence of 3G networks and increased capacity, the authors highlight the need for dynamic performance tracking across multi-provider environments.

Traffic Bottleneck Analysis: Babulak (2004) proposed a cost-effective methodology for detecting network bottlenecks in university infrastructure using systematic performance measurements. The approach enables organizations to identify congestion points, assess network behavior under load, and improve service quality.

Cloud-Based Security Experimentation: Bucur and Babulak (2019) developed a cloud-based experimental environment on Azure for validating theoretical cybersecurity models in practical settings. The framework enables realistic testing through scalable deployment.

6G in Healthcare: Kumar et al. (0000) explore the role of 6G technologies — such as ultra-low latency and high data rates — in enabling advanced medical applications, and propose an SHA-256 blockchain-based method for medical image protection.

QoS Evaluation: Babulak (2006) introduces a cost-effective methodology for assessing Quality of Service (QoS) from the user's perspective, demonstrating its applicability in healthcare communication environments.

QoS in Campus Networks: Babulak (2003) extends existing QoS assessment models by integrating user feedback with traffic measurements, offering a practical and inexpensive method for evaluating QoS in campus networks.

Secure and Energy-Efficient IoT: Mohanraj and Babulak (2019) developed an IoT system for wireless sensor networks using a Rectangular Rationale Correlated Bayesian method, enhancing encryption efficiency and reducing energy consumption.

6. Discussion

The proposed method, built upon the Temporal Graph Network (TGN) and its extension for edge classification (TGNE), offer an effective dynamic modeling framework for alert prediction by representing attacker-victim interactions as temporal bipartite graphs. A key innovation in our work is the introduction of a Transformer-based aggregator (TGNE-TA), replacing the original message aggregation function in TGN. This component leverages self-attention to learn complex interaction dependencies across a node's temporal neighborhood, significantly enhancing the model's ability to capture fine-grained and non-local temporal features. In addition, we integrate a Dual-Level Temporal Encoding mechanism, combining both absolute time encodings and positional encodings, within the aggregation phase. This design improves the model's temporal awareness by encoding both global temporal shifts and local sequence position, contributing to more accurate representation learning over evolving networks. Our model demonstrate strong performance in forecasting future attacks, accurately capturing the evolving temporal and structural patterns between attackers and victims. Our proposed TGNE-TA adds another layer of semantic understanding by predicting not only whether an interaction will occur but also its category (e.g., attack type). This dual-objective formulation enriches the output by providing actionable information for cybersecurity analysts, such as the predicted type of attack and its likely target. Unlike GRU and BiLSTM, temporal graph modeling captures relational dependencies across multiple entities rather than isolated sequences. Inference is performed locally over recent neighbors, enabling low-latency predictions without full history reprocessing, while periodic updates suffice instead of frequent retraining, making the approach suitable for real-time deployment. Despite these improvements, some limitations remain. The effectiveness of the models is still sensitive to the quality and granularity of input data. Incomplete logs, lack of fine-resolution timestamps, or imbalanced class distributions may hinder the models' generalization, particularly for rare or zero-day attack patterns. In zero-day scenarios, the framework can still predict attacker–victim interactions from temporal patterns, but accurate attack-type classification is limited without prior labels. Moreover, while the Transformer aggregator improves the model's expressiveness, it introduces additional computational overhead, which may affect real-time applicability on large-scale streaming data. Techniques such as memory compression or sparse attention could be explored to address this. Another challenge relates to the assumption of static feature semantics over time. As attacker strategies and network defenses co-evolve, there is a need for adaptive models capable of online learning and continual updating of learned representations. Furthermore, the model is subject to limitations related to dataset scope, closed-world assumptions of predefined attack categories, and generalization to unseen attacks and evolving network behaviors. Future work may consider incorporating reinforcement learning or continual learning frameworks for long-term deployment in adversarial environments. In terms of practical deployment, extending the model to fuse heterogeneous data sources — such as host logs, honeypot alerts, and external threat intelligence feeds — may provide a more holistic view of the attack surface. Additionally, integrating uncertainty estimation or explainability modules into the prediction pipeline would further enhance analyst trust and facilitate actionable use. In summary, our Transformer-enhanced TGN architecture with temporal dual encoding and edge classification capabilities offers a promising direction for temporal graph-based cyber-attack forecasting. False positives may cause unnecessary actions; thus, predictions should support expert decisions rather than operate autonomously.

7. Conclusion

In this work, we introduced a dynamic graph–based deep learning framework for alert prediction. Starting from a TGN for binary link prediction, we extended it to TGNE, enabling joint prediction of future attacker–victim interactions and their attack types. To enhance temporal reasoning, we integrated a Transformer-based message aggregator with a Dual-Level Temporal Encoding mechanism, allowing the model to capture both global and local temporal dynamics. Our experiments show that the proposed framework achieves consistent gains in link prediction accuracy and reliably forecasts upcoming attacks, target nodes, and attack categories in evolving network environments. Future work focuses on scaling to enterprise-level traffic and incorporating richer contextual signals to improve fine-grained attack classification.

Table 16
Alert prediction in related works.

	Prediction Works	Abbreviation	Explanation
1	Prediction of Alerts	PoA	Related alert features prediction
2	Prediction of Categorical	PoC	Prediction of classified features
3	Prediction of non-Categorical	PoN	Prediction of unclassified features
4	Predict with Alerts Correlation Graph	ACG	Prediction of correlation graph
5	Predict the upcoming steps of attack	PUS	Predicting the next steps of the attack
6	Prioritized the Alert data	PAD	Prioritizing alerts based on the importance of the time or intensity of the attack
7	Attack Prediction	ATP	Predicting the type of attack
8	Predictive Blacklist creation	PB	Blacklist prediction
9	Binary Prediction	BiP	Binary prediction (presence or absence of attack)
10	Attack Graph	AG	Attack graph construction
11	Vulnerability risks detection	VD	Prediction by vulnerability discovery
12	Real time Alert Prediction	RAP	Predicting the alerts as soon as the attack is expected or a step of the attack predicted
13	Personalization of the blacklist	PoB	Creating a blacklist appropriate for each organization
14	Predict targeting several networks at once	PSN	Anticipation targeting multiple networks at once
15	Novel Attack Prediction	NAP	Predicting a zero-day attack
16	Find associations between the various attack classes	ASA	Prediction based on attack type relationships

Table 17
Comparison of related works.

Paper	PoA	PoC	PoN	ACG	PUS	PAD	ATP	PB	BiP	AG	VD	RAP	PoB	PSN	NAP	ASA
Ansari et al. (2022)	✔	✔	✔	–	–	–	✔	–	–	–	–	–	–	–	–	–
Ansari et al. (2020)	✔	✔	✔	–	–	–	✔	–	–	–	–	–	–	–	–	–
Zhang et al. (2019)	✔	✔	✔	✔	✔	–	–	–	–	–	–	–	–	–	–	–
Bartos et al. (2019)	✔	–	✔	–	–	✔	✔	✔	✔	–	–	–	–	–	–	–
Liu (2020)	–	–	–	–	–	–	✔	–	–	✔	✔	–	–	–	–	–
Husák et al. (2020)	✔	✔	✔	–	–	–	–	✔	–	–	–	✔	✔	–	–	–
Husák et al. (2017)	✔	✔	✔	–	–	–	–	–	–	–	–	–	–	–	–	–
Husák and Kašpar (2018)	✔	✔	✔	–	–	–	–	–	–	–	–	–	–	✔	–	–
Husák et al. (2021)	✔	✔	✔	–	✔	✔	✔	✔	–	–	–	–	–	–	✔	–
Ben Fredj et al. (2020)	–	✔	–	–	–	–	✔	–	–	–	–	–	–	–	–	–
Melis et al. (2019)	–	–	–	–	–	–	✔	✔	–	–	–	–	–	–	–	–
Lei and Li (2007)	–	–	–	–	–	–	✔	–	–	✔	–	–	–	–	–	–
Li et al. (2007)	–	–	–	–	–	–	–	–	–	✔	–	–	–	–	–	✔
Hernández et al. (2016)	–	–	–	–	–	–	✔	–	–	–	–	–	–	–	–	–
Nayeri and Rezvani (2024)	✔	–	–	✔	–	–	–	–	✔	✔	–	–	–	✔	–	–
This work	✔	✔	✔	✔	–	–	✔	✔	✔	✔	–	✔	–	✔	–	–

CRediT authorship contribution statement

Zahra Makki Nayeri: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mohsen Rezvani:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Time complexity analysis

A.1. TGN

TGN generates temporal embeddings for nodes by aggregating messages from node neighborhoods in a dynamic manner.

1. Message Passing: Each layer performs multi-head attention across neighborhoods. For a graph with $|V|$ nodes and each node with n neighbors, the message-passing complexity is:

   $$\mathcal{O}(|V| \times L \times n \times H \times d^2)$$

   where d is the embedding dimension, L is the number of message-passing layers, and H is the number of attention heads.
2. Edge Probability Computation: Calculating probabilities for each edge requires $\mathcal{O}(|E| \times d)$ operations, assuming $|E|$ edges.

Overall Complexity:

$$\mathcal{O}(|V| \times L \times n \times H \times d^2 + |E| \times d)$$

A.2. TGN with edge classification

This variant extends TGN by also predicting edge categories, which enhances its application in tasks requiring classification of interactions in addition to predicting edge existence.

1. Category Prediction: Additional complexity of category prediction for each edge involves a linear layer:

   $\mathcal{O}(|E| \times d \times k)$

   where k is the number of categories.
2. Total Complexity: The complexity of TGN with edge classification remains dominated by the message-passing stage but now includes the edge classification layer:

   $\mathcal{O}(|V| \times L \times n \times H \times d^2 + |E| \times d \times k)$

A.3. Computational complexity of TGNE-TA

The computational complexity of the proposed TGNE model with a Transformer-based aggregator is driven by three primary components: neighborhood sampling and message construction, Transformer aggregation, and edge-level prediction. Let d denote the embedding dimension, n the number of sampled neighbors per node, and c the number of edge categories.

- **Message Construction:** For each interaction (u, v, t), the model samples up to n temporal neighbors of node v and computes message embeddings. This process scales linearly with the number of neighbors and the embedding size, leading to a complexity of $\mathcal{O}(n \cdot d)$.
- **Transformer Aggregation:** The Transformer aggregator operates over the sequence of message embeddings, applying self-attention and feed-forward layers. The attention mechanism incurs a quadratic cost in the sequence length and a linear or quadratic cost in embedding size. The overall time complexity of this component is:

  $\mathcal{O}(n^2 \cdot d + n \cdot d^2)$

  This makes the Transformer the computational bottleneck, especially with large n. In practice, neighborhood sampling is used to constrain n to small values (e.g., 10–20), ensuring scalability.
- **Edge Classification:** For each interaction, the model performs a binary prediction (via a dot-product) and a multi-class classification over edge types using a linear classifier. The combined cost of these steps is:

  $\mathcal{O}(d + d \cdot c)$

  which is negligible compared to the Transformer aggregation step.

Overall, the per-interaction time complexity is:

$\mathcal{O}(n^2 \cdot d + n \cdot d^2 + d \cdot c)$

This analysis confirms that while the Transformer aggregator introduces additional computational cost compared to simpler aggregation schemes (e.g., mean or attention pooling), it provides richer temporal modeling capabilities. By limiting the neighborhood size n, we maintain tractability even for large-scale temporal graphs.

Appendix B. TGNE parameter selection

Our initial experiment aimed to determine the best parameters for the TGNE-attention model in dynamic link prediction on an attack graph from the Warden alert dataset. We tested different settings, as shown in Table 18, including last vs. mean message aggregators, attention vs. sum embedding modules, GRU vs. RNN memory updaters, and identity vs. MLP message functions. For transductive tasks, we predicted links for known nodes, while for inductive tasks, we predicted links for new nodes. The optimal setup included last aggregator, attention embedding, GRU memory updater, and MLP message function.

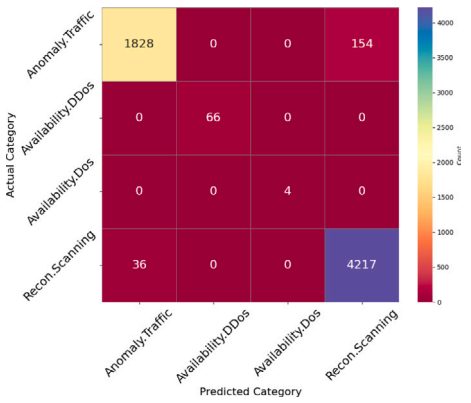


Fig. 16. TGNE confusion matrix in a week using GRU memory updater.

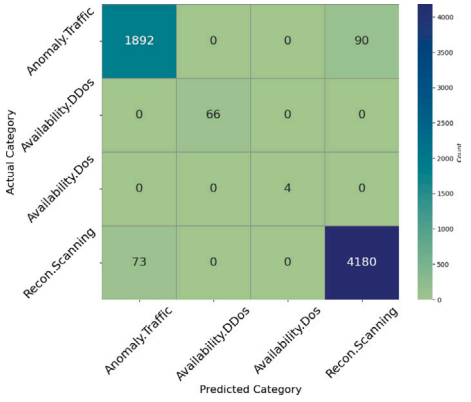


Fig. 17. TGNE confusion matrix in a week using RNN memory updater.

Appendix C. Confusion matrix

As shown in Figs. 16 and 17, we compared the confusion matrices for the GRU and RNN memory updates across four different types of attacks. The results indicate that the GRU memory updater outperforms the RNN in accurately predicting the Anomaly.Traffic category.

Appendix D. Hyperparameters

For all experiments, we use the Adam optimizer with a learning rate of 0.0001 and a batch size of 256 for training, validation, and testing. We use early stopping with a patience of 5 based on the validation loss. For link prediction, we sample an equal number of negative interactions for each positive one, following standard practice in temporal graph learning. Node embedding dimension is 128, time embedding dimension is 100, number of attention heads is 2 and dropout is 0.1 and number of epochs is 50. Multi-class category prediction is trained using the standard PyTorch CrossEntropyLoss with no class weighting; therefore, the alpha coefficient equals 1 for all classes. To ensure robustness under class imbalance, the decision threshold for binary edge prediction is not fixed. Instead, we sweep thresholds in the range [0.01, 0.99] and select the value that maximizes the F1-score on the validation set, which jointly penalizes false positives and false negatives. This data-driven threshold selection prevents bias toward the majority negative class. All experiments use neighbor sampling, where each node aggregates information from the k most recent neighbors, which we find to improve efficiency without reducing accuracy. All results are averaged over six independent runs with fixed random seeds to ensure robustness and reproducibility.

Table 18
Model evaluation metrics for multiple variants of TGNE.

Parameters	Type	AUC		Average Precision		MRR		Accuracy		Loss	
		Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive	Transductive
Message Aggregator	Last	0.9248	0.8601	0.9334	0.8552	0.9641	0.9342	0.9904	0.9863	0.0054	0.0990
	Mean	0.9136	0.8379	0.9285	0.8495	0.9689	0.9364	0.9873	0.9791	0.0088	0.0972
Embedding Module	Attention	0.9511	0.9035	0.9585	0.9083	0.9717	0.9485	0.9902	0.9859	0.0060	0.1323
	Sum	0.9005	0.8007	0.9115	0.8537	0.9590	0.9019	0.9621	0.9463	0.0654	0.1824
Memory Updator	GRU	0.9314	0.8830	0.9321	0.8793	0.9620	0.9390	0.9891	0.9825	0.0064	0.1565
	RNN	0.9207	0.8670	0.9041	0.8349	0.9570	0.9346	0.9909	0.9863	0.0023	0.1495
Message Function	Identity	0.5494	0.6498	0.5349	0.6356	0.7774	0.8268	0.8713	0.8293	0.3757	0.4657
	MLP	0.9392	0.8970	0.9477	0.8997	0.9678	0.9471	0.9823	0.9803	0.0092	0.1185

Data availability

The source code and datasets supporting this study are publicly available at: https://github.com/ZahraMakkiNayeri/TGNE-TA.git.

References

Anli, Y.A., Ciplak, Z., Sakaliuzun, M., Izgu, S.Z., Yildiz, K., 2024. DDoS detection in electric vehicle charging stations: A deep learning perspective via CICEV2023 dataset. Internet Things 28, 101343.

Ansari, M.S., Bartos, V., Lee, B., 2020. Shallow and deep learning approaches for network intrusion alert prediction. Proc. Comput. Sci. 171, 644–653.

Ansari, M.S., Bartoš, V., Lee, B., 2022. GRU-based deep learning approach for network intrusion alert prediction. Future Gener. Comput. Syst. 128, 235–247, URL https://www.researchgate.net/publication/355133237.

Babulak, E., 2003. Quality of service provision assessment for campus network. In: SympoTIC'03. Joint 1st Workshop on Mobile Future and Symposium on Trends in Communications. IEEE, pp. 75–78.

Babulak, E., 2004. Traffic bottleneck analysis in telecommunications infrastructures. In: SympoTIC'04. Joint 1st Workshop on Mobile Future & Symposium on Trends in Communications (IEEE Cat. No. 04EX877). IEEE, pp. 154–157.

Babulak, E., 2006. Quality of service provision assessment in the healthcare information and telecommunications infrastructures. Int. J. Med. Inform. 75 (3–4), 246–252.

Babulak, E., 2023. AI Tools for Protecting and Preventing Sophisticated Cyber Attacks. IGI Global.

Bartos, V., Zadnik, M., Habib, S.M., Vasilomanolakis, E., 2019. Network entity characterization and attack prediction. Future Gener. Comput. Syst. 97, 674–686.

Ben Fredj, O., Mihoub, A., Krichen, M., Cheikhrouhou, O., Derhab, A., 2020. CyberSecurity attack prediction: a deep learning approach. In: 13th International Conference on Security of Information and Networks. pp. 1–6.

Bucur, C., Babulak, E., 2019. Security validation testing environment in the cloud. In: 2019 IEEE International Conference on Big Data (Big Data). IEEE, pp. 4240–4247.

Buyuktanir, B., Altinkaya, Ş., Karatas Baydogmus, G., Yildiz, K., 2025. Federated learning in intrusion detection: advancements, applications, and future directions. Clust. Comput. 28 (7), 1–25.

Büyüktanir, B., Çıplak, Z., Çil, A.E., Yakar, Ö., Adoum, M.B., Yıldız, K., 2025. DDoS_FL: Federated learning architecture approach against DDoS attack. Pamukkale Üniversitesi Mühendislik Bilim. Derg. 31 (6).

Chung, C.-J., Khatkar, P., Xing, T., Lee, J., Huang, D., 2013. NICE: Network intrusion detection and countermeasure selection in virtual network systems. IEEE Trans. Depend. Secur. Comput. 10 (4), 198–211.

Cil, A.E., Yildiz, K., Buldu, A., 2021. Detection of DDoS attacks with feed forward based deep neural network model. Expert Syst. Appl. 169, 114520.

Hernández, A., Sanchez, V., Sánchez, G., Pérez, H., Olivares, J., Toscano, K., Nakano, M., Martinez, V., 2016. Security attack prediction based on user sentiment analysis of Twitter data. In: 2016 IEEE International Conference on Industrial Technology. ICIT, IEEE, pp. 610–617.

Hughes, T., Sheyner, O., 2003. Attack scenario graphs for computer network threat analysis and prediction. Complexity 9 (2), 15–18.

Husák, M., Bajtoš, T., Kašpar, J., Bou-Harb, E., Čeleda, P., 2020. Predictive cyber situational awareness and personalized blacklisting: a sequential rule mining approach. ACM Trans. Manag. Inf. Syst. (TMIS) 11 (4), 1–16.

Husák, M., Bartoš, V., Sokol, P., Gajdoš, A., 2021. Predictive methods in cyber defense: Current experience and research challenges. Future Gener. Comput. Syst. 115, 517–530.

Husák, M., Kašpar, J., 2018. Towards predicting cyber attacks using information exchange and data mining. In: 2018 14th International Wireless Communications & Mobile Computing Conference. IWCMC, IEEE, pp. 536–541.

Husák, M., Kašpar, J., Bou-Harb, E., Čeleda, P., 2017. On the sequential pattern and rule mining in the analysis of cyber security alerts. In: Proceedings of the 12th International Conference on Availability, Reliability and Security. pp. 1–10.

Husák, M., Komárková, J., Bou-Harb, E., Čeleda, P., 2018. Survey of attack projection, prediction, and forecasting in cyber security. IEEE Commun. Surv. & Tutorials 21 (1), 640–660.

Kacha, P., Kostenec, M., Kropacova, A., 2015. Warden 3: Security event exchange redesign. In: Proceedings of the 19th International Conference on Computers: Recent Advances in Computer Science.

Kazemi, S.M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., Brubaker, M., 2019. Time2vec: Learning a vector representation of time. arXiv preprint arXiv:1907.05321.

Kotenko, I., Chechulin, A., 2013. A cyber attack modeling and impact assessment framework. In: 2013 5th International Conference on Cyber Conflict (CYCON 2013). IEEE, pp. 1–24.

Kumar, S., Joy, J., Jane, I.C., James, A., John, S., Babulak, E., 0000. Applications of 6G in the healthcare sector and the importance of network security and data privacy-encryption of medical images using the SHA 256 blockchain algorithm, In: Network Security and Data Privacy in 6G Communication, CRC Press, pp. 176–190.

Lei, J., Li, Z.-t., 2007. Using network attack graph to predict the future attacks. In: 2007 Second International Conference on Communications and Networking in China. IEEE, pp. 403–407.

Li, T., Jiang, Y., Lin, C., Obaidat, M.S., Shen, Y., Ma, J., 2023. DeepAG: Attack graph construction and threats prediction with bi-directional deep learning. IEEE Trans. Dependable Secur. Comput. 20 (1), 740–757. http://dx.doi.org/10.1109/TDSC.2022.3143551.

Li, Z.-t., Lei, J., Wang, L., Li, D., 2007. A data mining approach to generating network attack graph for intrusion prediction. In: Fourth International Conference on Fuzzy Systems and Knowledge Discovery (FSKD 2007). Vol. 4, IEEE, pp. 307–311.

Liu, X., 2020. A network attack path prediction method using attack graph. J. Ambient. Intell. Humaniz. Comput. 1–8.

Massam, Babulak, E., 2003. A decent service level. Commun. Eng. 1 (3), 34–37.

Melis, L., Pyrgelis, A., De Cristofaro, E., 2019. On collaborative predictive blacklisting. ACM SIGCOMM Comput. Commun. Rev. 48 (5), 9–20.

Mohanraj, R., Babulak, E., 2019. A secure energy efficient IoT based fractional correlated Bayesian data transmission in WSNs. J. Commun. Inf. Netw. 4 (1), 54–66.

Nayeri, Z.M., Rezvani, M., 2024. Alert prediction in computer networks using deep graph learning. In: 2024 10th International Conference on Signal Processing and Intelligent Systems. ICSPIS, IEEE, pp. 1–5.

Nia, M.A., Atani, R.E., Fabian, B., Babulak, E., 2016. On detecting unidentified network traffic using pattern-based random walk. Secur. Commun. Netw. 9 (16), 3509–3526.

Okutan, A., Yang, S.J., McConky, K., 2017b. Predicting cyber attacks with bayesian networks using unconventional signals. In: Proceedings of the 12th Annual Conference on Cyber and Information Security Research. pp. 1–4.

Qin, X., Lee, W., 2004. Attack plan recognition and prediction using causal networks. In: 20th Annual Computer Security Applications Conference. IEEE, pp. 370–379.

Ramaki, A.A., Khosravi-Farmad, M., Bafghi, A.G., 2015. Real time alert correlation and prediction using Bayesian networks. In: 2015 12th International Iranian Society of Cryptology Conference on Information Security and Cryptology. ISCISC, IEEE, pp. 98–103.

Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., Bronstein, M., 2020. Temporal graph networks for deep learning on dynamic graphs. arXiv preprint arXiv:2006.10637.

Sendi, A.S., Dagenais, M., Jabbarifar, M., Couture, M., 2012. Real time intrusion prediction based on optimized alerts with hidden Markov model. J. Netw. 7 (2), 311.

Shen, Y., Mariconti, E., Vervier, P.A., Stringhini, G., 2018. Tiresias: Predicting security events through deep learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 592–605.

Sokol, P., Gajdoš, A., 2018. Prediction of attacks against honeynet based on time series modeling. In: Applied Computational Intelligence and Mathematical Methods: Computational Methods in Systems and Software 2017, Vol. 2. Springer, pp. 360–371.

Wang, Y., Li, J., Zhao, W., Han, Z., Zhao, H., Wang, L., He, X., 2023. N-STGAT: Spatio-temporal graph neural network based network intrusion detection for near-earth remote sensing. Remote. Sens. 15 (14), 3611.

Werner, G., Yang, S., McConky, K., 2017. Time series forecasting of cyber attack intensity. In: Proceedings of the 12th Annual Conference on Cyber and Information Security Research. pp. 1–3.

Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., Achan, K., 2020. Inductive representation learning on temporal graphs. arXiv preprint arXiv:2002.07962.

Zhan, Z., Xu, M., Xu, S., 2013. Characterizing honeypot-captured cyber attacks: Statistical framework and case study. IEEE Trans. Inf. Forensics Secur. 8 (11), 1775–1789.

Zhang, Z., Shen, H., 2005. Application of online-training SVMs for real-time intrusion detection with different considerations. Comput. Commun. 28 (12), 1428–1442.

Zhang, K., Zhao, F., Luo, S., Xin, Y., Zhu, H., 2019. An intrusion action-based IDS alert correlation analysis and prediction framework. IEEE Access 7, 150540–150551.

Zhao, J., Tang, T., Bu, B., Li, Q., 2024. Graph neural network-based attack prediction for communication-based train control systems. CAAI Trans. Intell. Technol.

Zahra Makki Nayeri is a Ph.D. candidate at Shahrood University of Technology and currently a visiting researcher at the University of Stuttgart, Germany, where she conducts research on knowledge graph foundation models. Her research focuses on graph representation learning, temporal and dynamic graph neural networks, and machine learningbased modeling of computer networks, with an emphasis on data-driven analysis of large-scale, evolving interaction graphs. During her Master's studies, she investigated machine learning techniques for fog, edge, and cloud computing environments, with a particular focus on distributed data processing, resource-aware learning, and system-level optimization.

Mohsen Rezvani is an Associate Professor in the Faculty of Computer Engineering at the Shahrood University of Technology. He received his Ph.D. at the School of Computer Science and Engineering at the UNSW Sydney. He holds an M.Sc. in Computer Engineering from Sharif University of Technology and a B.Sc. in Computer Engineering from Amirkabir University of Technology. His research focuses on computer security, intelligent data analysis, and advanced machine learning. He is the director of the Computer Emergency Response Team (CERT) at Shahrood University of Technology.